

MODULE 4 -Tree

BCSE202L - DSA

Module:4 Trees

Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.

Definition

- A tree is a non linear data structure that represents hierarchical relationships between individual data items
- A tree is a collection of nodes. The collections can be empty; otherwise, a tree consists of a distinguished node r called the root, and zero or more nonempty subtrees $T_1, T_2, \dots, T_k$, each of whose roots are connected by a directed edge from r .

- The root of each subtree is said to be a child of r , and r is the parent of subtree root.
- From the recursive definition, we find that a tree is a collection of n nodes, one of which is the root, and $n - 1$ edges.
- That there are $n - 1$ edges follows from the fact that each edge connects some node to its parent, and every node except the root has one parent

- A tree is a finite set of one or more nodes such that: There is a specially designated node called the root.
- The remaining nodes are partitioned into disjoint sets $T_1, \dots, T_n$, where each of these sets is a tree.
- A tree is a graph $G(V,E)$ without any cycle.

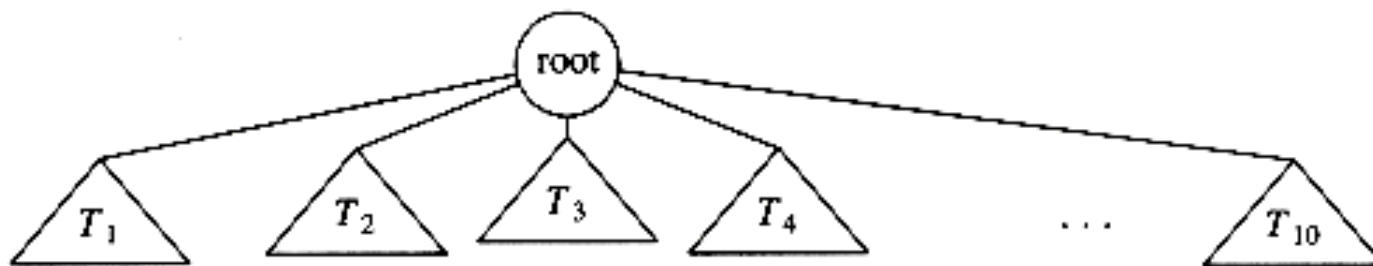


Figure 1 Generic tree

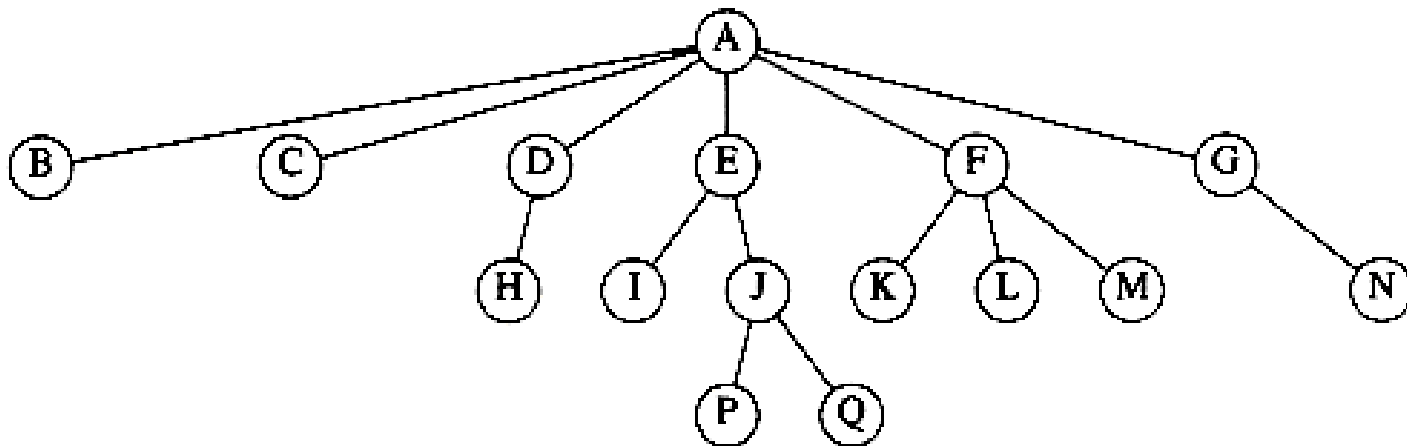


Figure 2 Tree

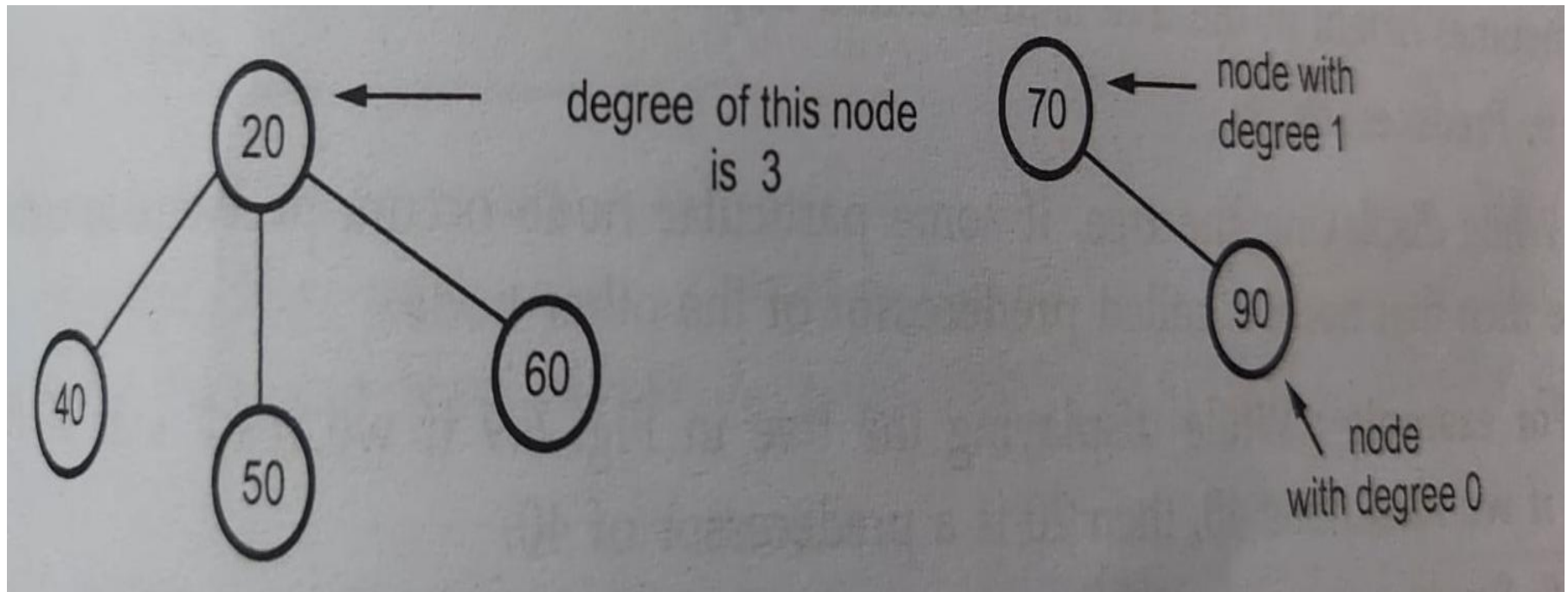
- In the tree of Figure 2,
 - the root is A.
 - Node F has A as a parent and K, L, and M as children.
 - Each node may have an arbitrary number of children, possibly zero.
 - Nodes with no children are known as leaves; the leaves in the tree above are B, C, H, I, P, Q, K, L, M, and N.
 - Nodes with the same parent are siblings; thus K, L, and M are all siblings.
 - Grandparent and grandchild relations can be defined in a similar manner.

BASIC Terminologies

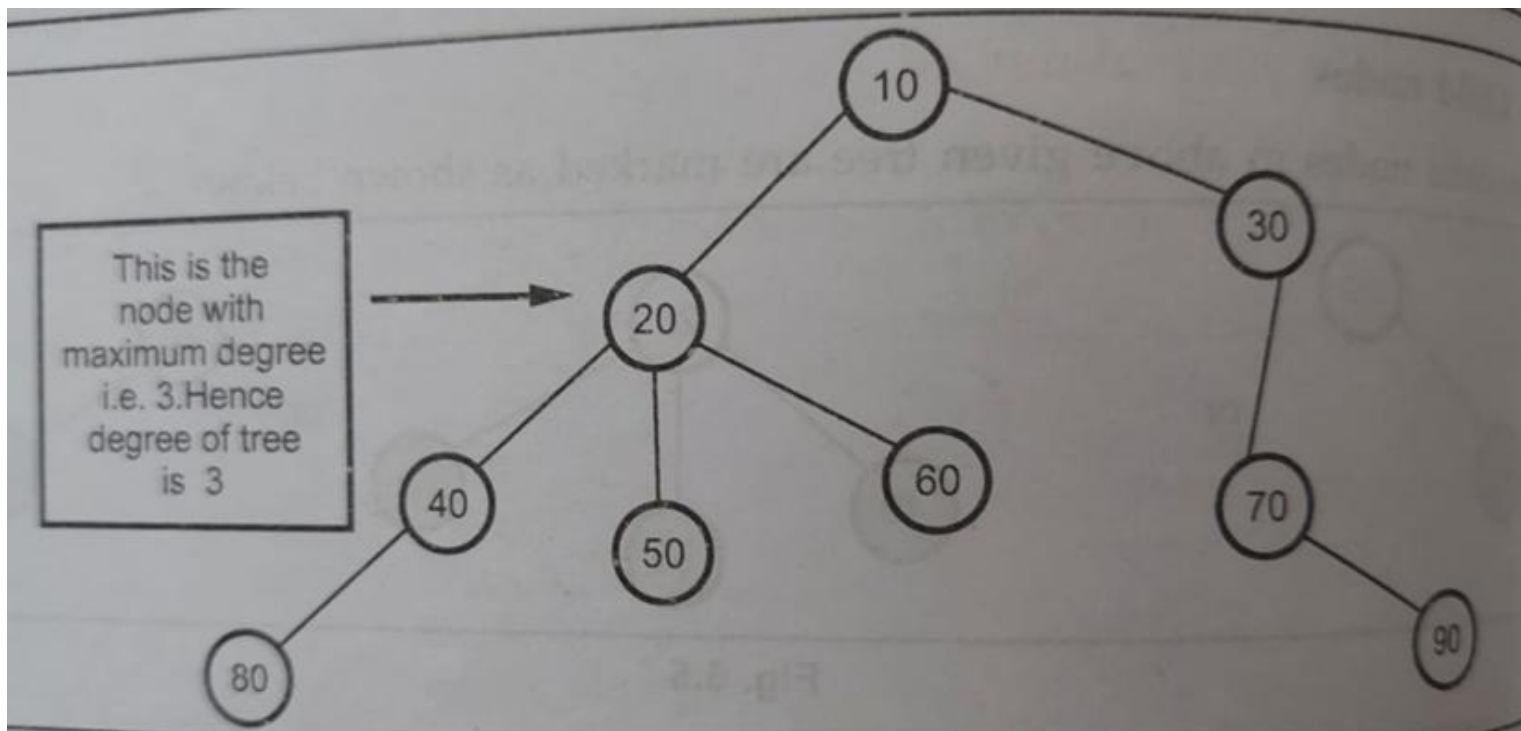
- Root node :- Top most node in the tree is root node and its usually called as parent node
- Parent node – Node having further sub branches is called parent node.
- Child node: - Nodes that are directly connected to root node are called as child nodes
- Siblings:- Child nodes of the same parent are called siblings
- Branch:- A link between a parent and child is called branch
- Ancestor : - The root node is the ancestor of all the nodes
- Leaf:- The node does not have any child node is said to be the leaf node. These nodes are the terminal nodes or external nodes of the tree.
- Subtree:- Parent nodes below the root node is said to be separate subtree

Degree

- Degree of a node:- Is defined as the number of sub-trees under a node



Degree of the Tree: The maximum degree in the tree is degree of tree.



- Edge: it is a connecting line of two nodes.
- Path: it is a sequence of consecutive edges from the source node to the destination node.
- Predecessor: while displaying the tree, if some particular node occurs prior to some other nodes then that node is called predecessor
- Successor: is a node which occurs next to some node.
- Forest: it is a set of disjoint trees

- **Non-Terminal node (Internal):** Any node (except the root node) whose degree is not zero is called non-terminal node. Non-terminal nodes are intermediate nodes while traversing the given tree from its root node to the terminal nodes (leaves).
- **Level :** The entire tree structure is levelled in such a way that the root node is always at level 0. Then its immediate children are at level 1 and their immediate children are at level 2 and so on upto the terminal nodes. In general, if a node is at level n , then its children will be at level $n+1$.
- **Depth of the tree:-** the maximum level is the depth of the tree. It always starts in reverse order (0 from root).

IMPLEMENTATION OF TREES

- One way to implement a tree is to have, a pointer to each child of the node.
- However, since the number of children per node can vary so greatly and is not known in advance, it might be infeasible to make the children direct links in the data structure, because there would be too much wasted space.
- Solution:
Keep the children of each node in a linked list of tree nodes

Node declarations for trees

```
struct tree_node  
{  
    int element;  
    struct tree_node * first_child;  
    struct tree_node * next_sibling;  
} *tree_ptr;
```

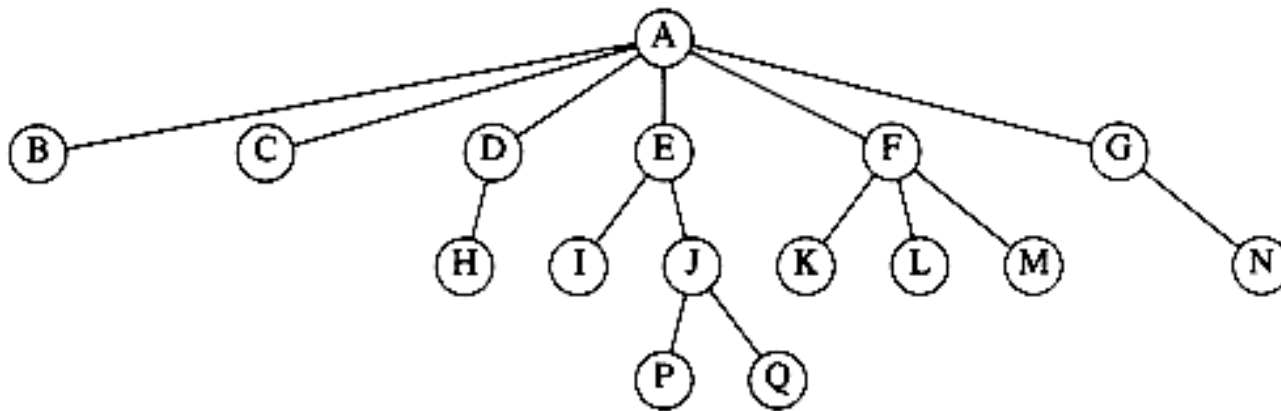


Figure 3 A tree

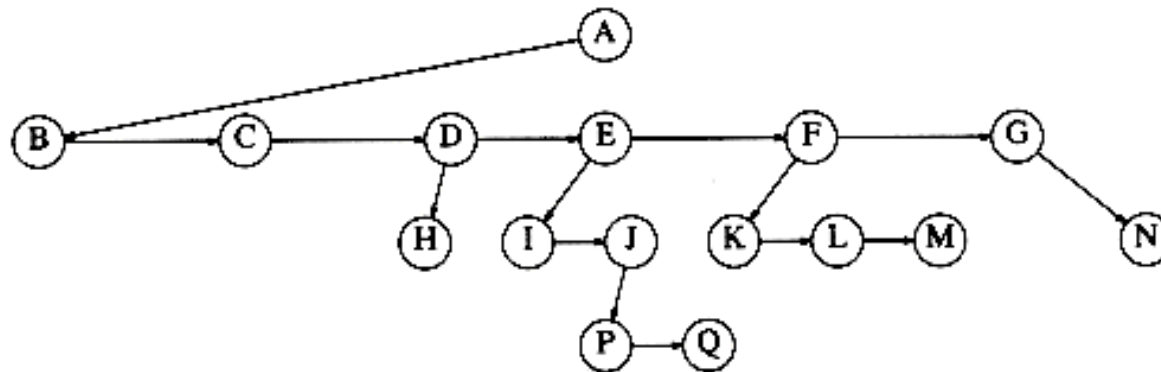
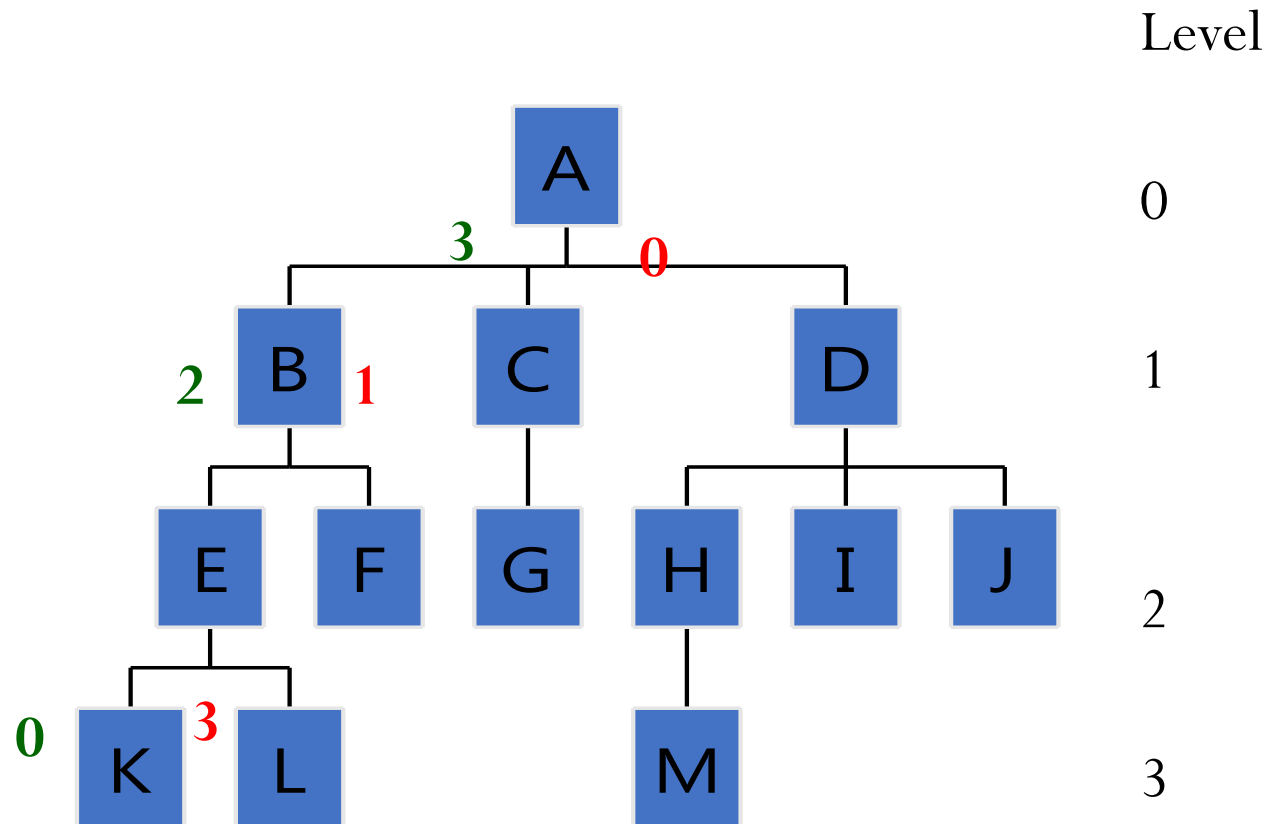


Figure 4 : First child/next sibling representation of the tree shown in above Figure

Tree example



Tree example

- **A** is the root node
- A is the parent to all the node and it's the ancestor to all
- A has B,C,D as child node and B,C,D are siblings
- B, C, D E,H are subtrees
- K,L,F,G, I, J are leaf nodes
- Degree of A is 3. depth of A is 0
- Degree of B is 2 and depth is 1
- Degree of K is 0 and depth is 3

Applications of tree

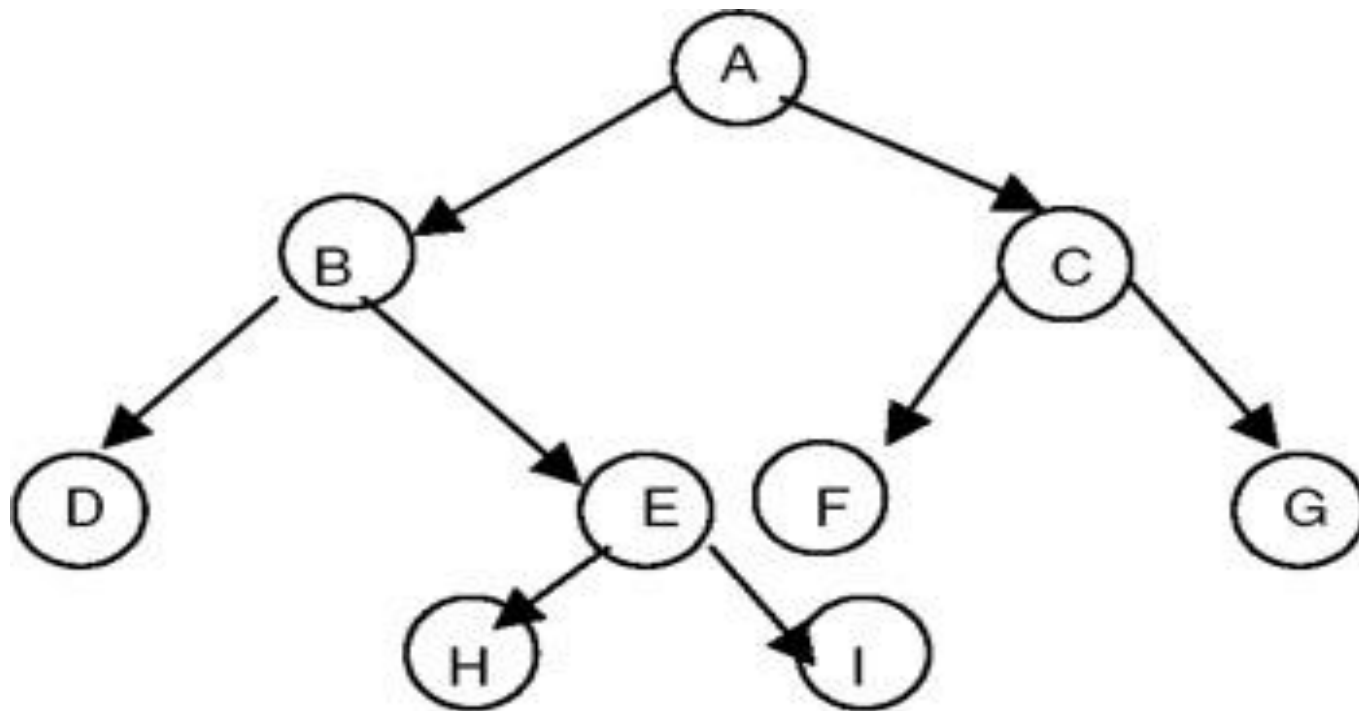
- Used in both computer science and data processing application
- File index system, hierarchical data base management, information management system.
- The directory structure of a file system on a computer
- Parse trees in compilers

Binary Trees

- In Binary tree each node has no more than two child nodes
- A binary tree is a finite set of nodes that is either empty or consists of a root and two disjoint binary trees called *the left subtree* and *the right subtree*.
- It is characterized by the fact that any node can have at most two branches, i.e., there is no node with degree greater than two

- A binary tree is a 2-ary tree, i.e. each node in the tree can have a maximum of 2 subtrees- that is it can have either 0,1, or 2 subtrees.
- A formal definition is:
- A binary tree is a finite collection of elements in which one element is designated as root and remaining elements are partitioned into two disjoint sets which are trees on their own (each of these sets may be null/ non-null), called left and right subtrees.

Binary Trees



For the given Binary Tree, find

- Root
- Parent node
- Siblings
- Leaf nodes
- Ancestor nodes
- Descendant node
- Left descendant
- Right descendant
- Path
- Length of the path
- Level of a node
- Depth of tree

Types of Binary Tree

- **Strict/ Proper Binary trees:**
 - Each node has either 0 or 2 child nodes
- **Perfect / Complete Binary Trees :**
 - All internal nodes have 2 children and all leaf nodes are at same level.
- **Almost Complete Binary Trees:**
 - The nodes at all levels are filled with maximum nodes (except last level). In the last levels the nodes are filled as left as possible.
 - Max. no of nodes = $2^{h+1}-1$ where h is the height
 - Min. no of nodes = 2^h
- **Degenerate Binary Trees:**
 - All internal node can have one child

Strictly Binary Tree

- It is a binary tree where all the nodes will have either 0 or 2 children
- [GIVE EXAMPLE]

Complete Binary Tree

- It is a binary tree in which all leaves are at the last level. A complete binary tree of depth d will have all its leaves at depth d , ie all levels of the tree will be full. [GIVE EXAMPLE]

Almost Complete Binary Tree

- An almost complete binary tree of depth d will have its leaves at level d or $d-1$. All levels upto level $d-1$ will be full with maximum number of nodes and level d will be filled from left to right.
- [GIVE EXAMPLE]

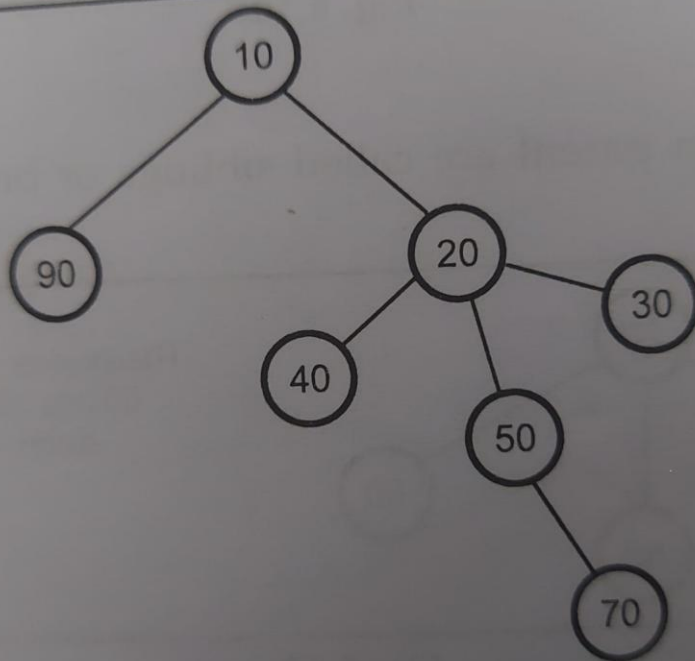
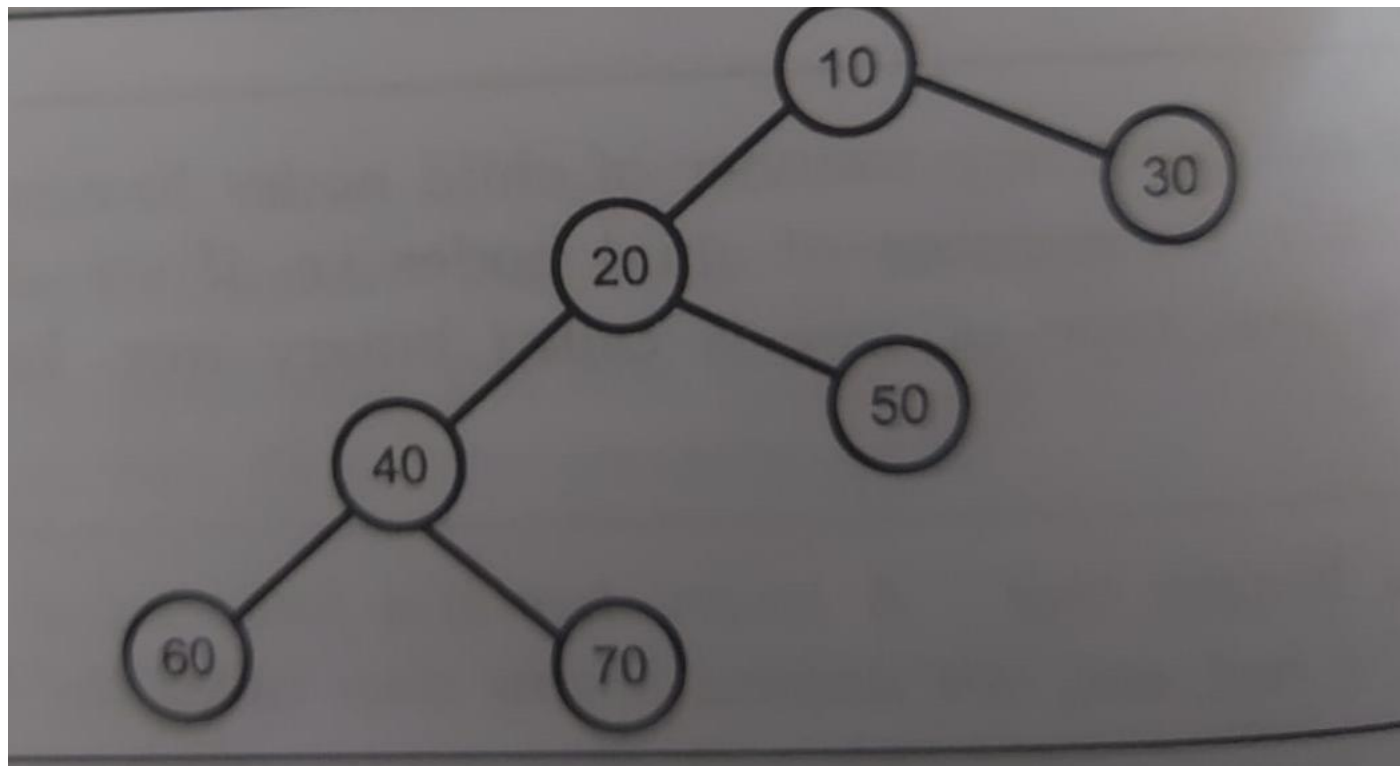
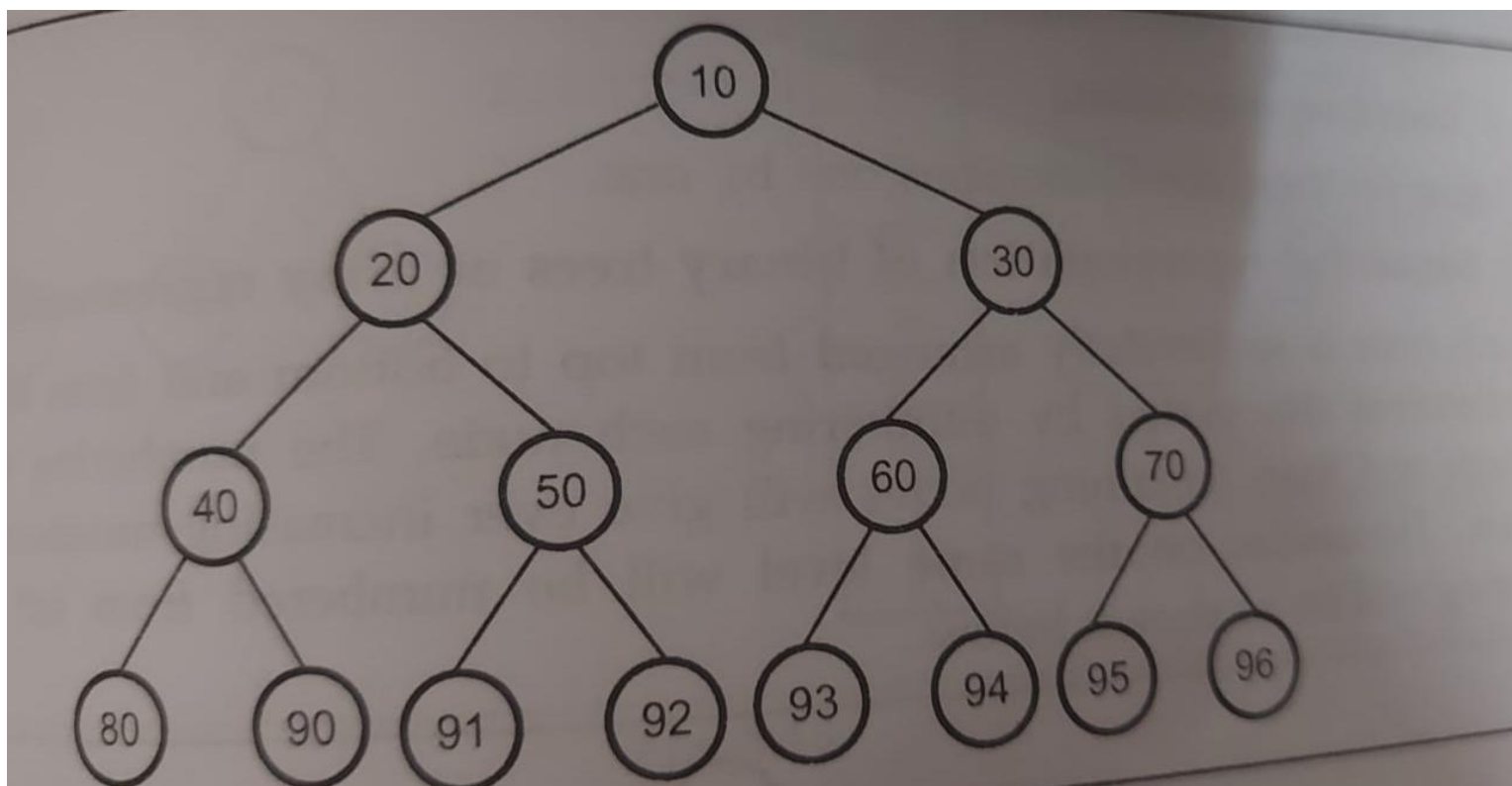


Fig. 6.13 Not a binary tree

Tree

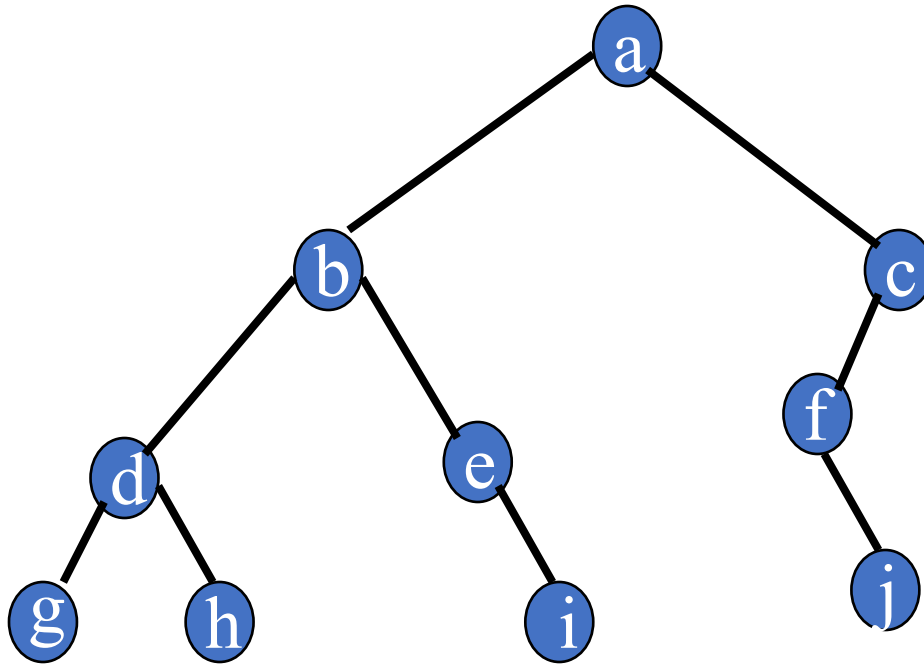




Binary trees, implementing Binary trees

- linear representation of binary tree
- Linked representation of binary tree

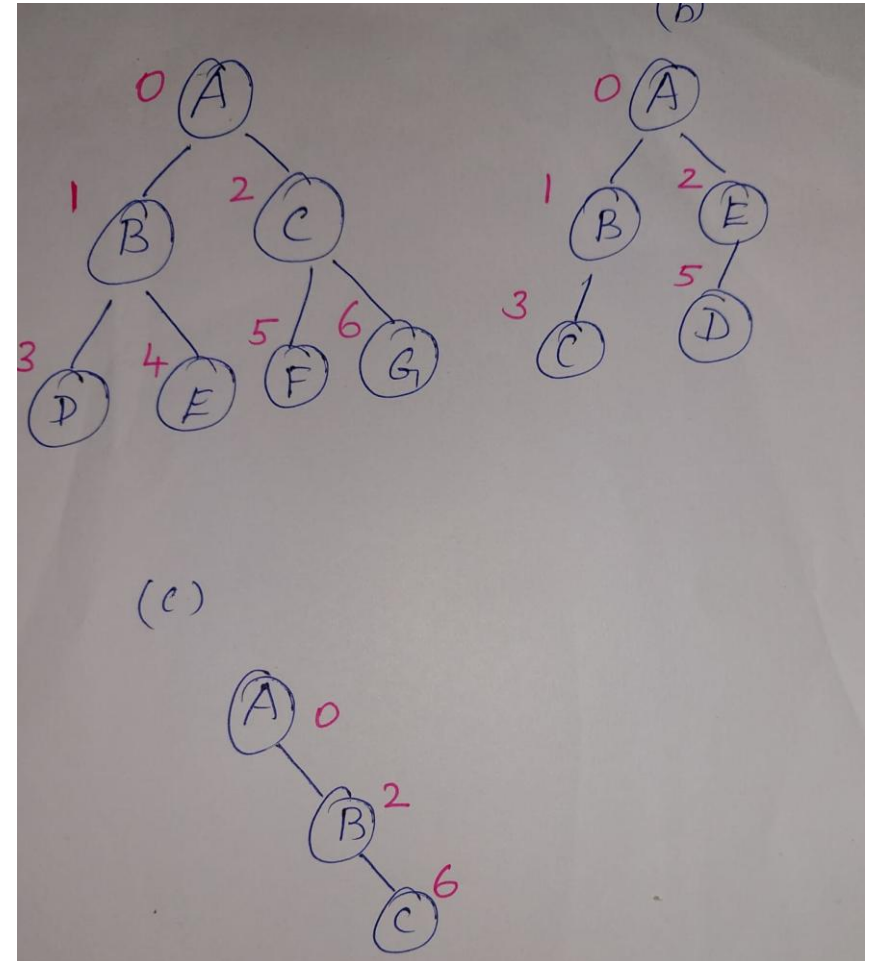
linear representation of binary tree,

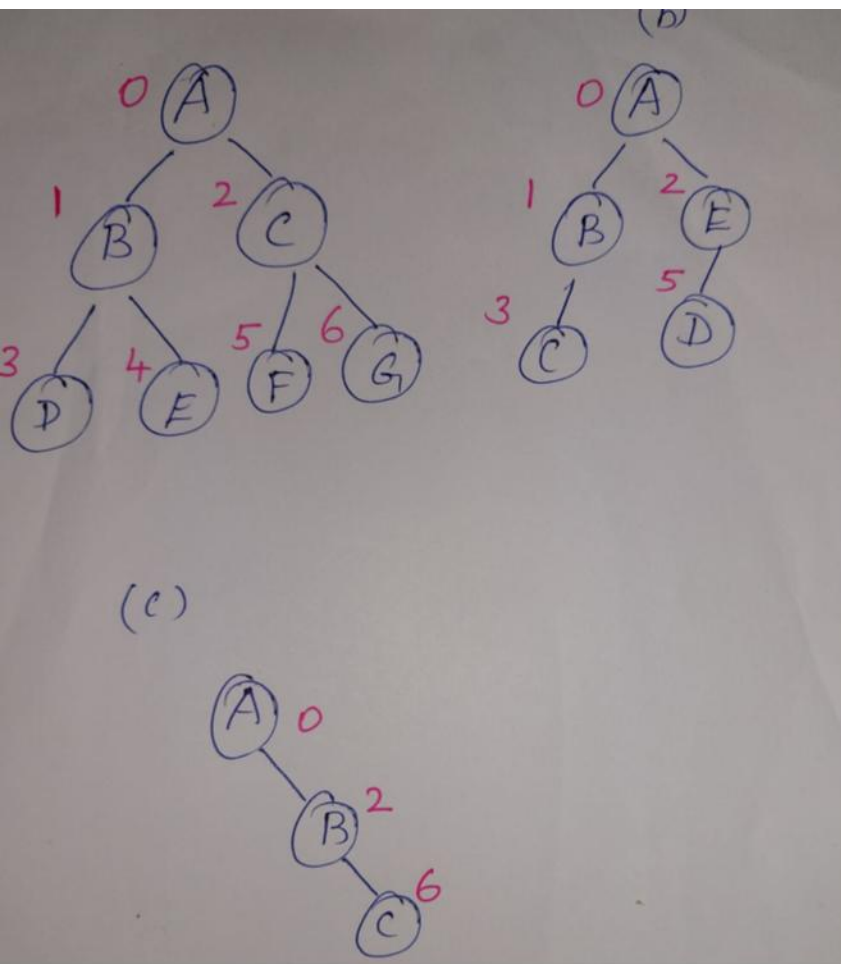


a
b
c
d
e
f
g
h
i
j

Linear representation

- In this representation, nodes are numbered from 0 from level by level (left to right)
- When a subtree is empty, **the numbers are also skipped**
- A Binary tree of depth 'd' is represented by using one-dimensional array of size $2^{(d+1)} - 1$.





index	value
0	A
1	B
2	C
3	D
4	E
5	F
6	G

index	value
0	A
1	B
2	E
3	C
4	
5	D
6	

index	value
0	A
1	
2	B
3	
4	
5	
6	C

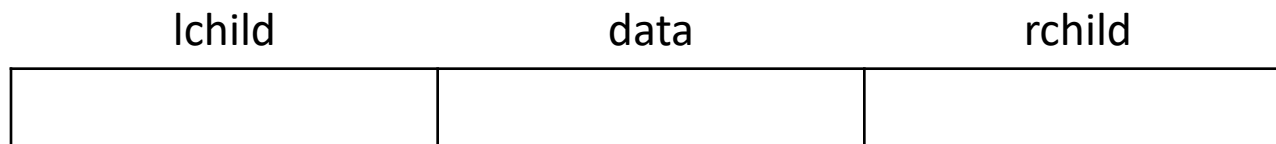
Note:

- Left child of node at index p is at index $2*p+1$ and right child is at $2*p + 2$. In other words, p is the parent index of nodes at $2*p+1$ and $2*p+2$.
- If left child is at index ' lc ' , its sibling will be at ' $lc+1$ '
- If right child is at index ' rc ' its sibling will be at ' $rc-1$ '
- Parent of a node at index ' c ' is at $\text{floor}((c-1)/2)$.

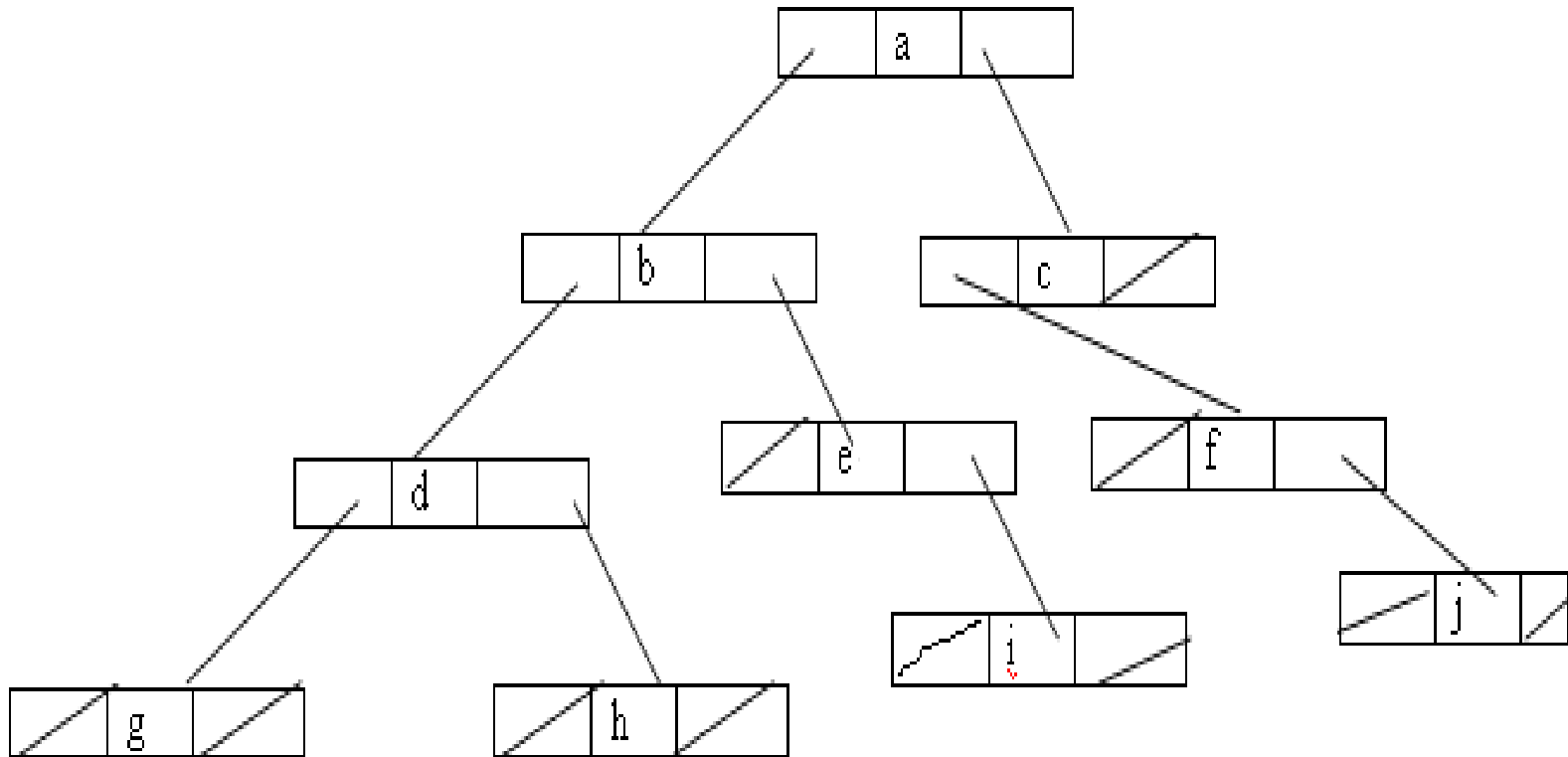
Linked Representation

- Use linked lists to represent binary trees
- Node structure in C language

```
struct BT_node  
{  
    int data;  
    struct BT_node *lchild;  
    struct BT_node *rchild;  
};
```



Linked representation of binary tree



Create a node in a binary tree

```
struct BT_node NewNode;  
NewNode=(struct BT_node*) malloc(sizeof(struct BT_node));  
NewNode->data=x;  
NewNode->lchild=NULL;  
NewNode->rchild=NULL;  
return NewNode;
```

binary tree traversals, Pre-order, In-order, Post-order traversals of binary tree

- In a traversal of a binary tree, each element of the binary tree is visited exactly once.
- During the visit of an element, all action (make a clone, display, evaluate the operator, etc.) with respect to this element is taken.

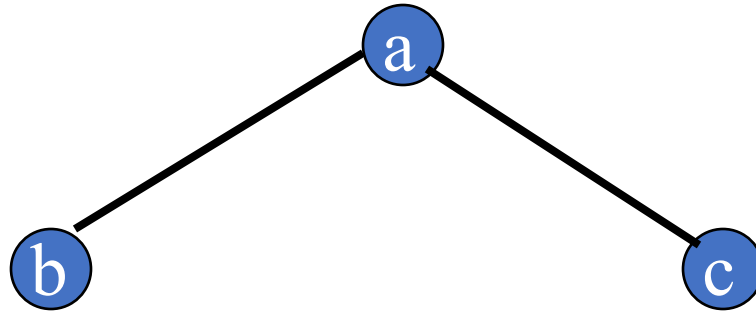
Binary Tree Traversal Methods

- Preorder
- Inorder
- Postorder
- Converse Preorder
- Converse Inorder
- Converse Postorder
- Level order

Preorder Traversal

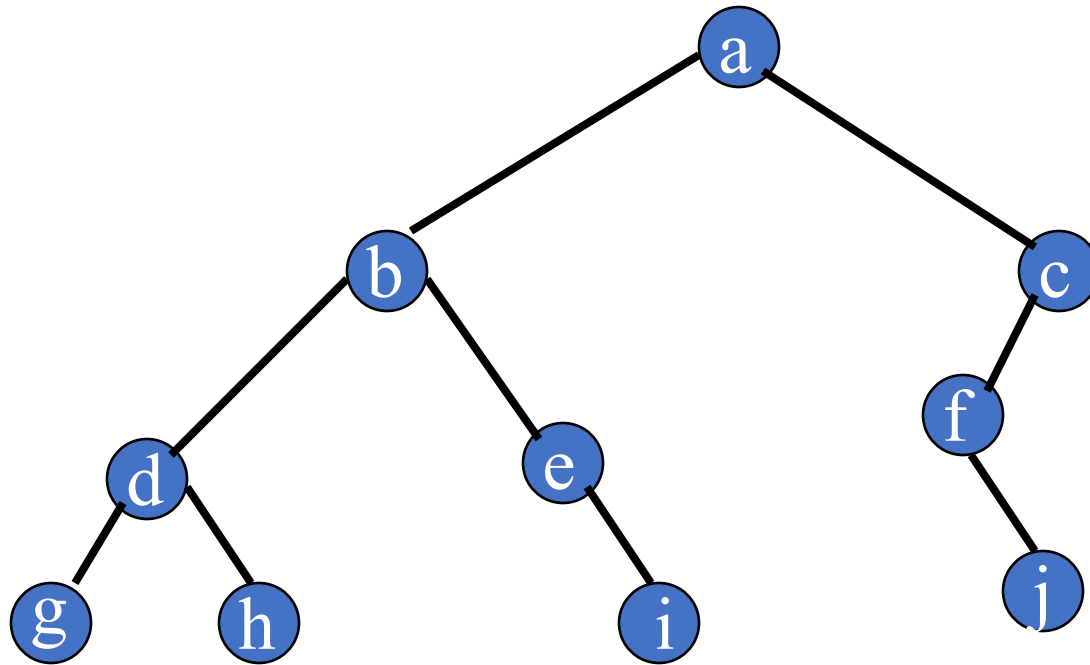
```
preOrder(BinaryTreeNode t)
{
    (t != null)
    {
        visit(t);
        preOrder(t.leftChild);
        preOrder(t.rightChild);
    }
}
```

Preorder Example (visit = print)



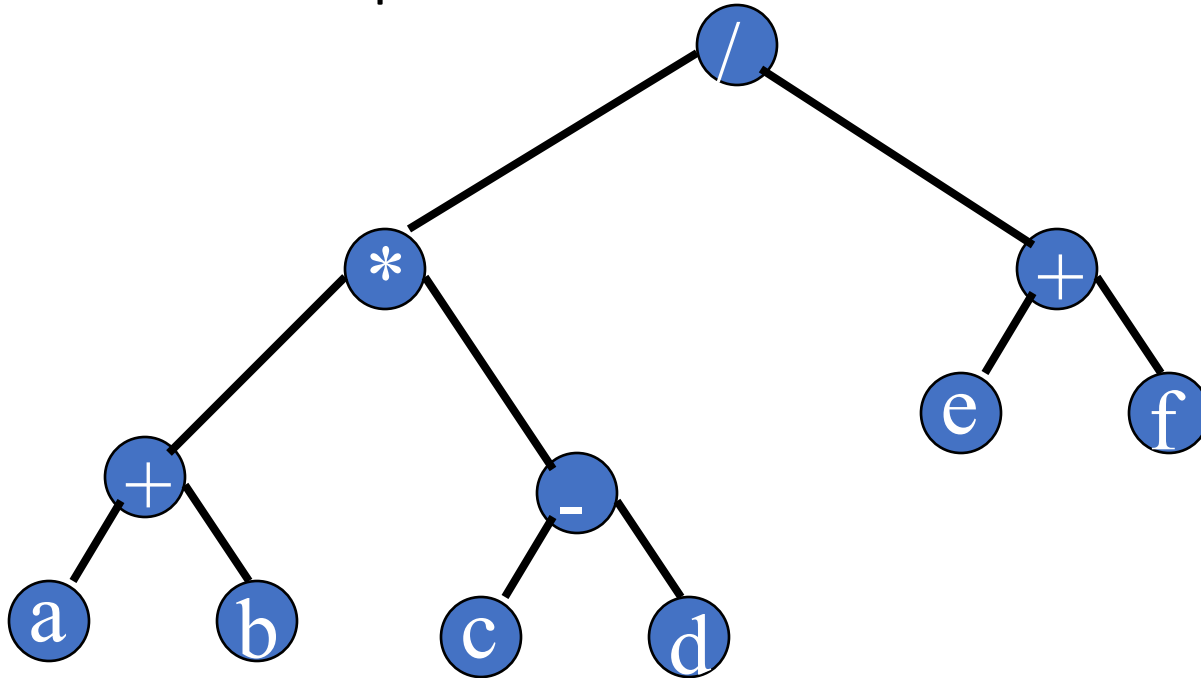
a b c

Preorder Example (visit = print)



a b d g h e i c f j

Preorder Of Expression Tree



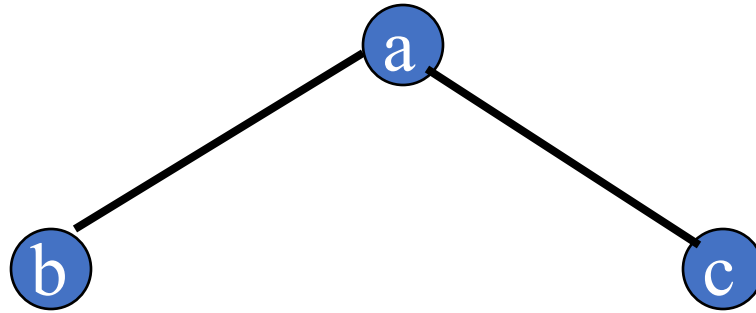
$/ * + a b - c d + e f$

Gives prefix form of expression!

Inorder Traversal

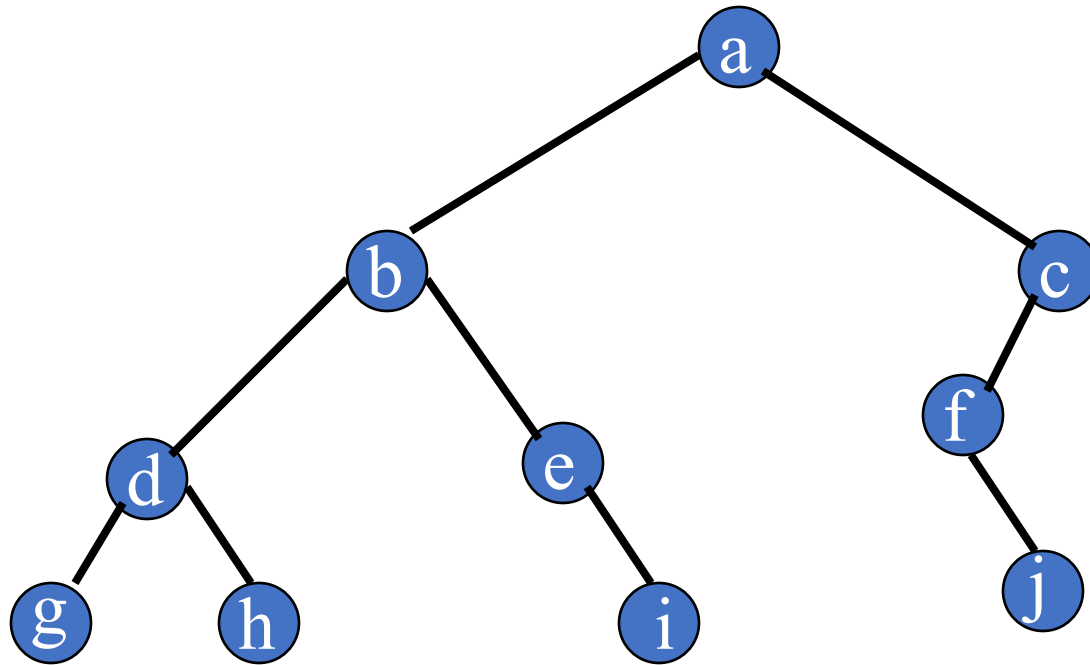
```
inOrder(BinaryTreeNode t)
{
    (t != null)
    {
        inOrder(t.leftChild);
        visit(t);
        inOrder(t.rightChild);
    }
}
```

Inorder Example (visit = print)



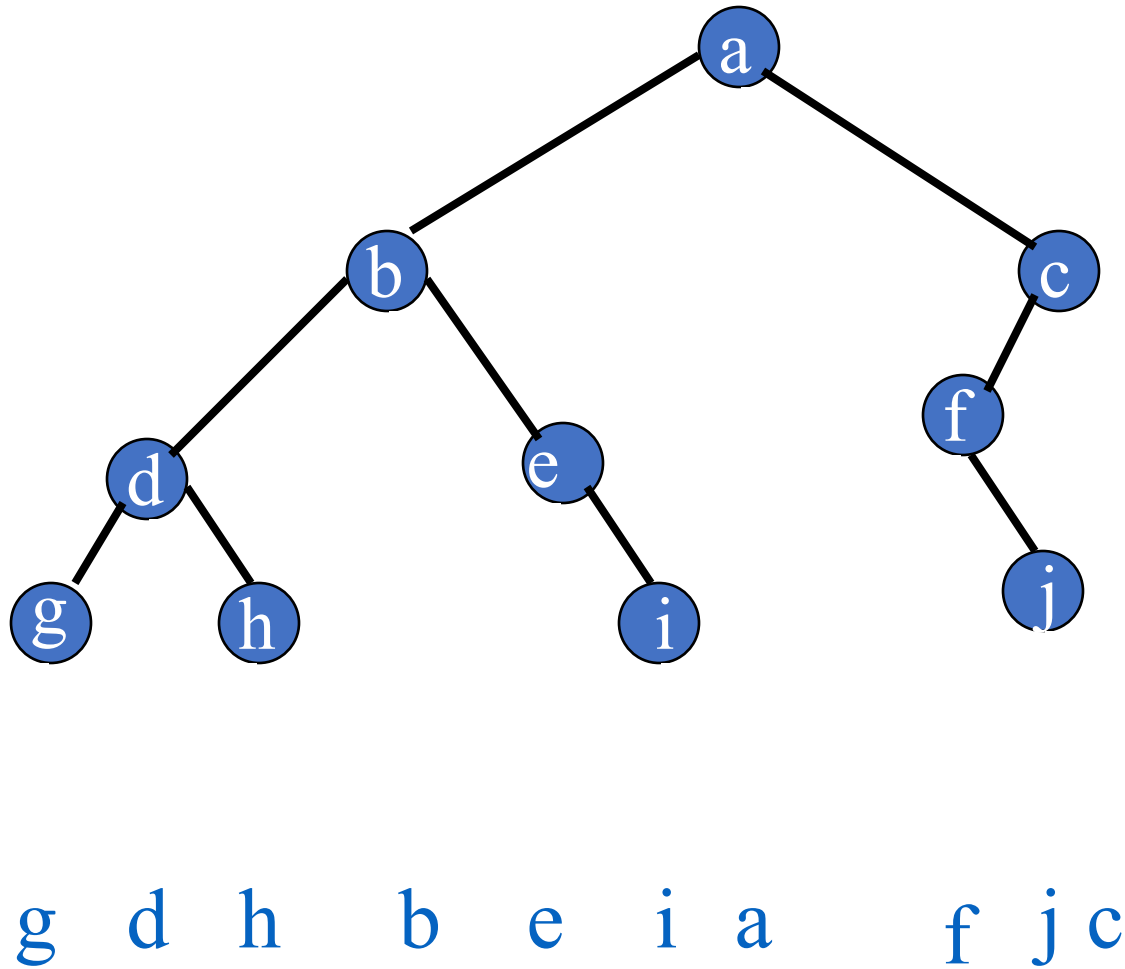
b a c

Inorder Example (visit = print)

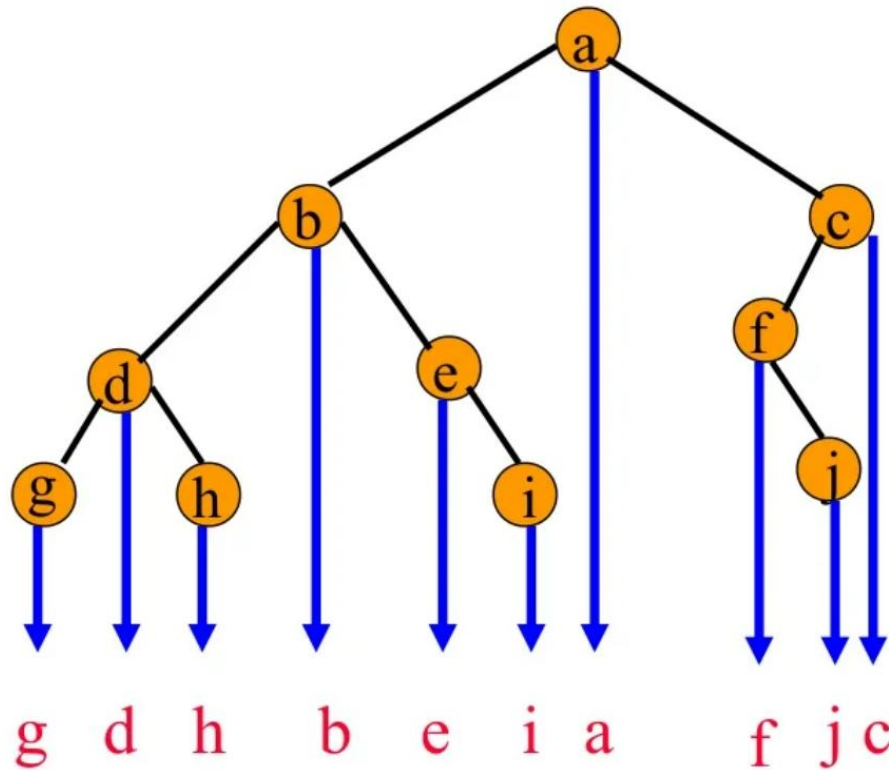


g d h b e i a f j c

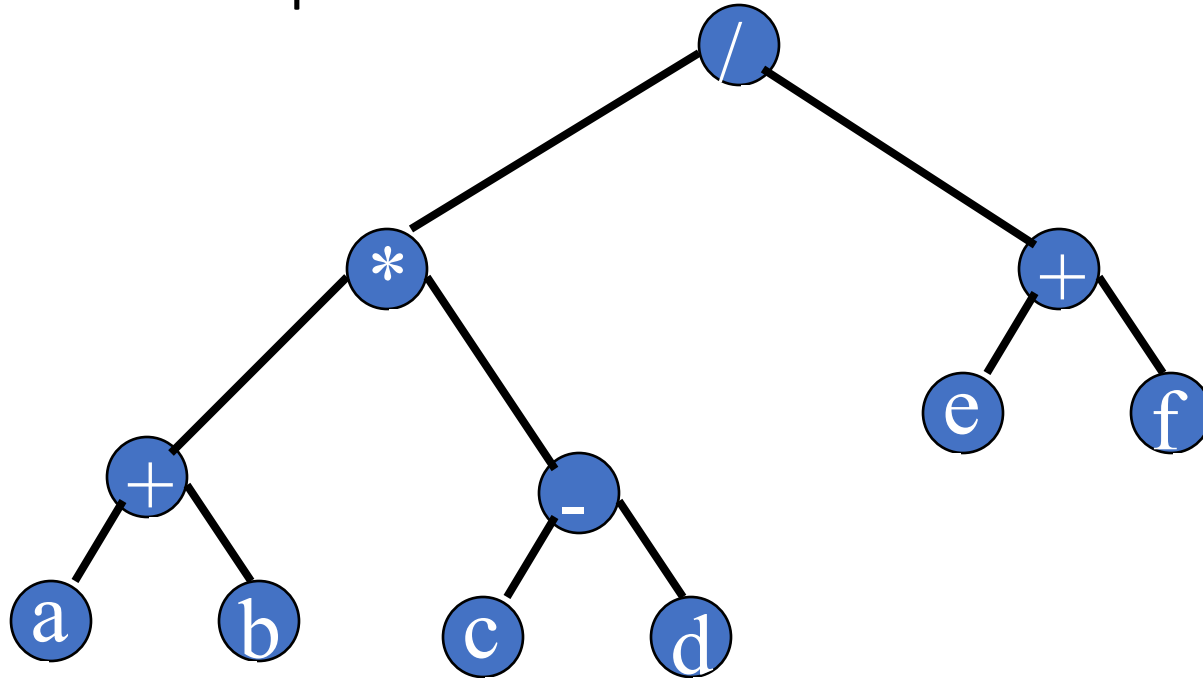
Inorder By Projection (Squishing)



Inorder By Projection (Squishing)



Inorder Of Expression Tree



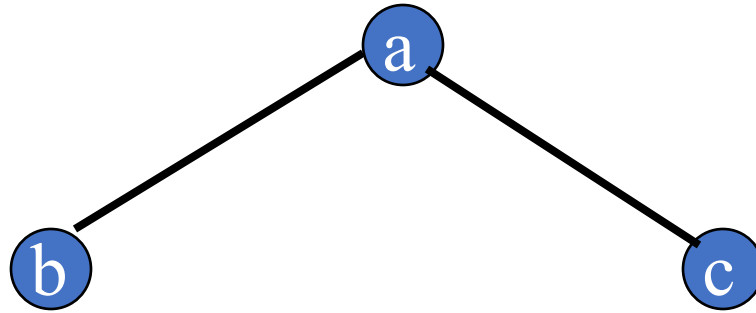
$a + b * c - d / e + f$

Gives infix form of expression

Postorder Traversal

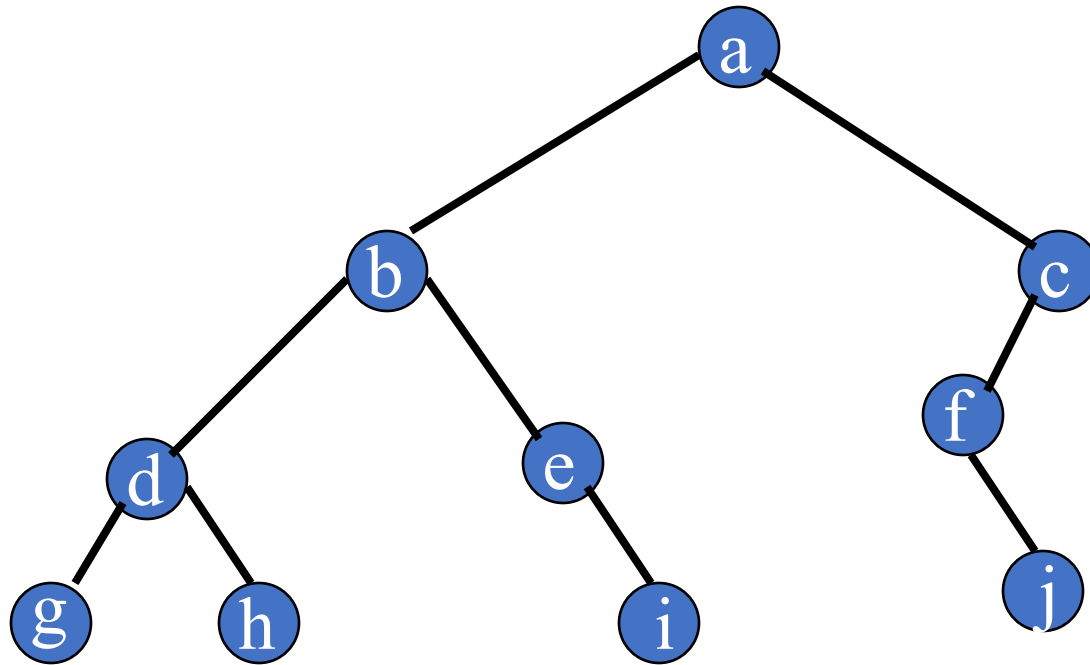
```
        postOrder(BinaryTreeNode t)
{
    (t != null)
    {
        postOrder(t.leftChild);
        postOrder(t.rightChild);
        visit(t);
    }
}
```

Postorder Example (visit = print)



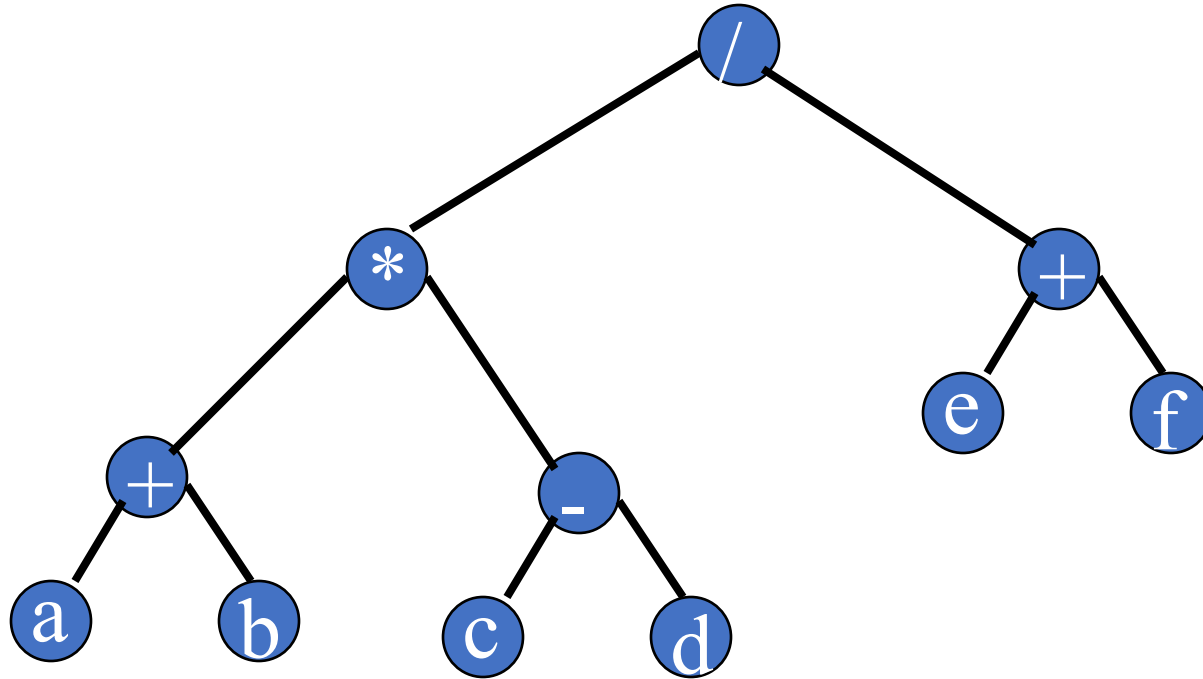
b c a

Postorder Example (visit = print)



g h d i e b j f c a

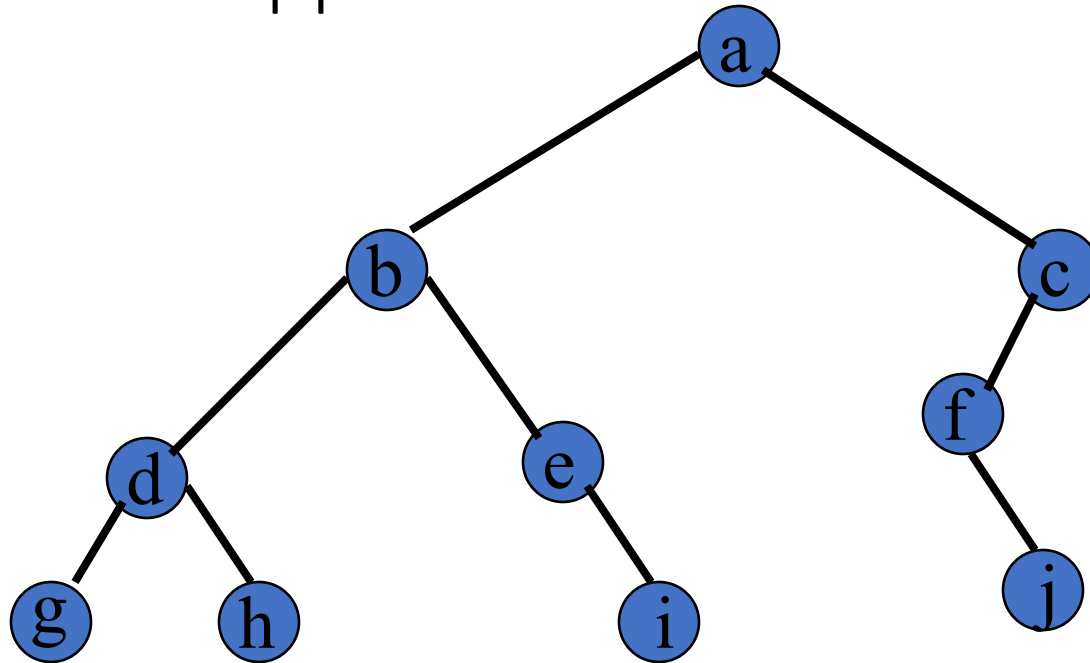
Postorder Of Expression Tree



a b + c d - * e f + /

Gives postfix form of expression!

Traversal Applications



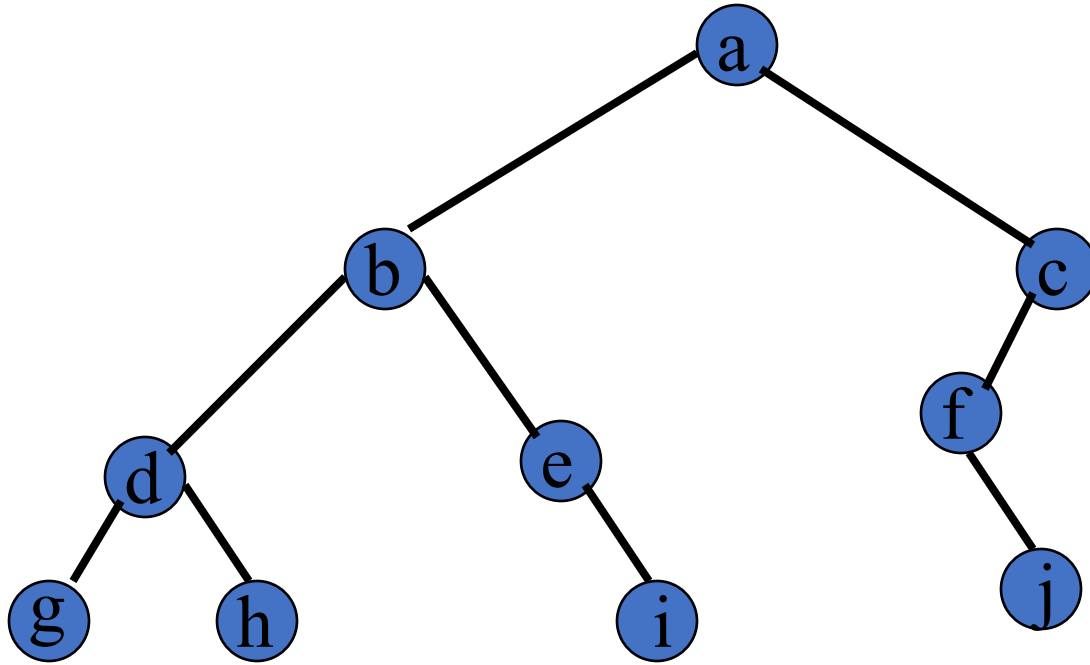
- Make a clone.
- Determine height.
- Determine number of nodes.

Level Order

Let **t** be the tree root.

```
(t != null)
{
    visit t and put its children on a FIFO queue;
    remove a node from the FIFO queue and call it t;
    // remove returns null when queue is empty
}
```

Level-Order Example (visit = print)

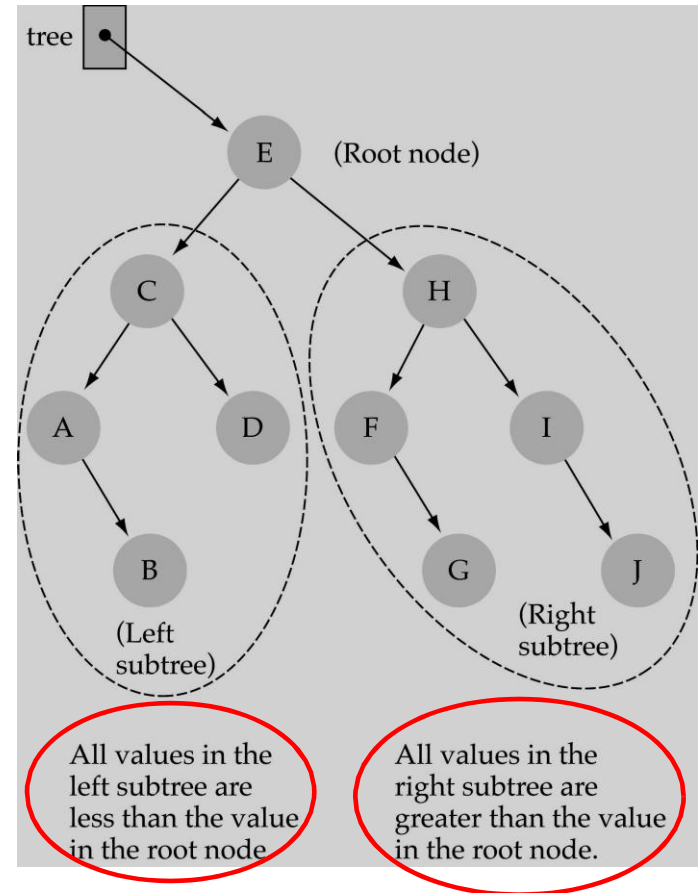


a b c d e f g h i j

Binary Search Trees

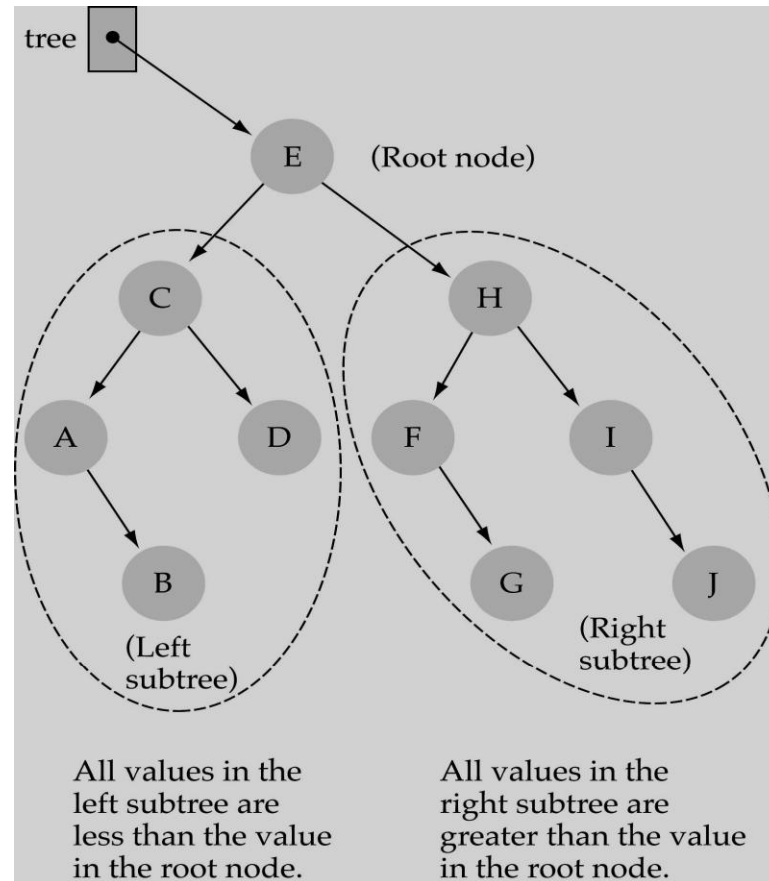
- **Binary Search Tree Property:**

The value stored at a node is *greater* than the value stored at its left child and *less* than the value stored at its right child



Binary Search Trees

In a BST, the value stored at the root of a subtree is *greater* than any value in its left subtree and *less* than any value in its right subtree!



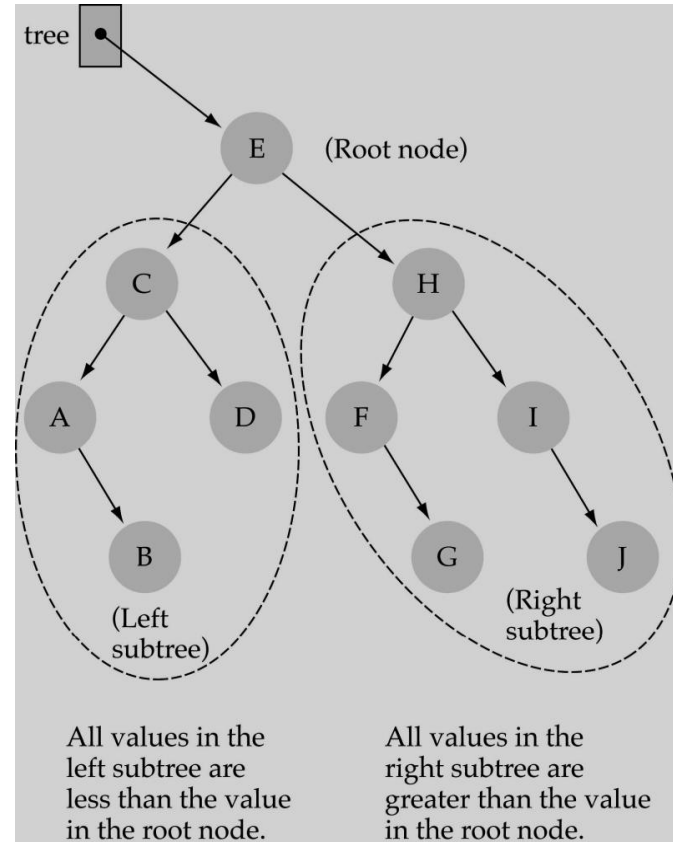
Binary Search Trees

Where is the smallest element?

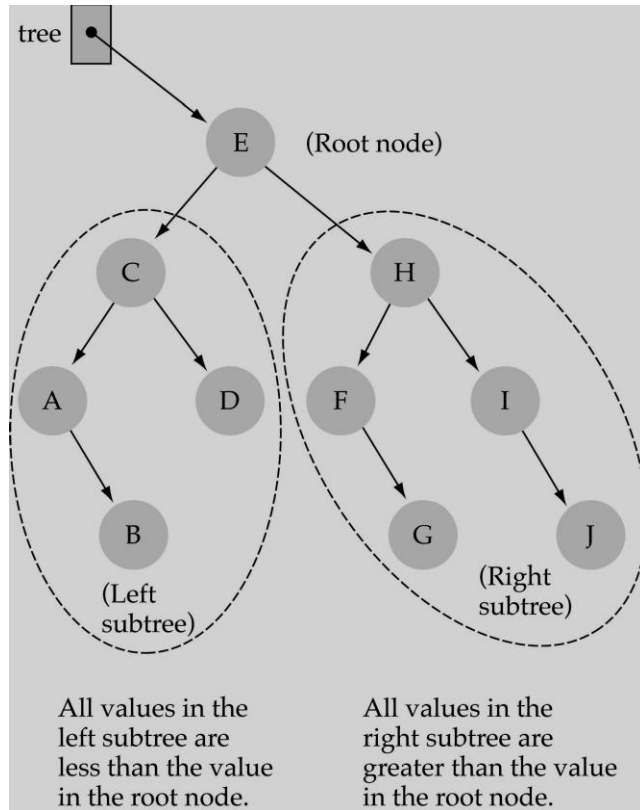
Ans: leftmost element

Where is the largest element?

Ans: rightmost element

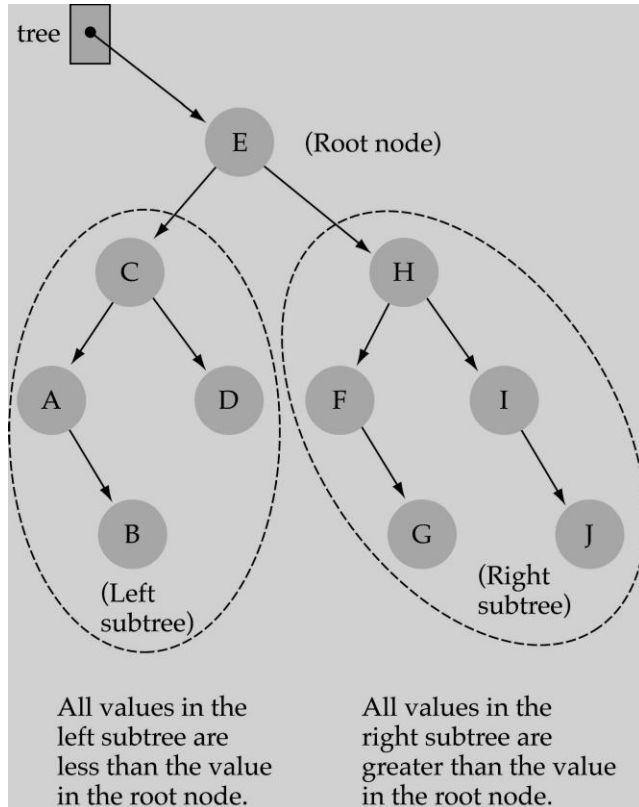


How to search a binary search tree?



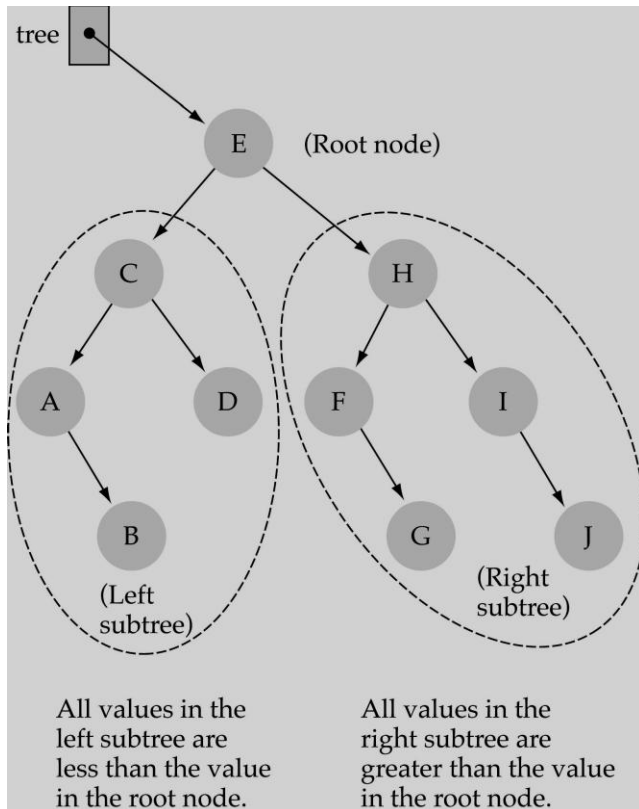
1. Start at the root
2. Compare the value of the item you are searching for with the value stored at the root
3. If the values are equal, then *item found*; otherwise, if it is a leaf node, then *not found*

How to search a binary search tree?



4. If it is less than the value stored at the root, then search the left subtree
5. If it is greater than the value stored at the root, then search the right subtree
6. Repeat steps 2-6 for the root of the subtree chosen in the previous step 4 or 5

How to search a binary search tree?



Is this better than searching a linked list?

Yes !!



0

(log N)

Difference between BT and BST

- A binary tree is simply a tree in which each node can have at most two children.
- A binary search tree is a binary tree in which the nodes are assigned values, with the following restrictions :
 1. No duplicate values.
 2. The left subtree of a node can only have values less than the node
 3. The right subtree of a node can only have values greater than the node and recursively defined
 4. The left subtree of a node is a binary search tree.
 5. The right subtree of a node is a binary search tree.

BST Search Algorithm

1. Let x be a node in a binary search tree and k is the value, we are supposed to search.
2. Then according to the binary search tree property we know that: if y is a node in the left subtree of x , then $y.key \leq x.key$. If y is a node in the right subtree of x , then $y.key \geq x.key$. ($x.key$ denotes the value at node x)
3. To search the location of given data k , the binary search tree algorithm begins its search at the root and traces the path downward in the tree.
4. For each node x it compares the value k with $x.key$. If the values are equal then the search terminates and x is the desired node.

BST Search Algorithm

5. If k is smaller than $x.key$, then the search continues in the left subtree of x , since the binary search tree property implies that k could not be in the right subtree.
6. Symmetrically, if k is larger than $x.key$, then the search continues in the right subtree.
7. The nodes encountered during the recursion form a simple path downward from the root of the tree, and thus the running time of TREE-SEARCH is $O(h)$, where h is the height of the tree.

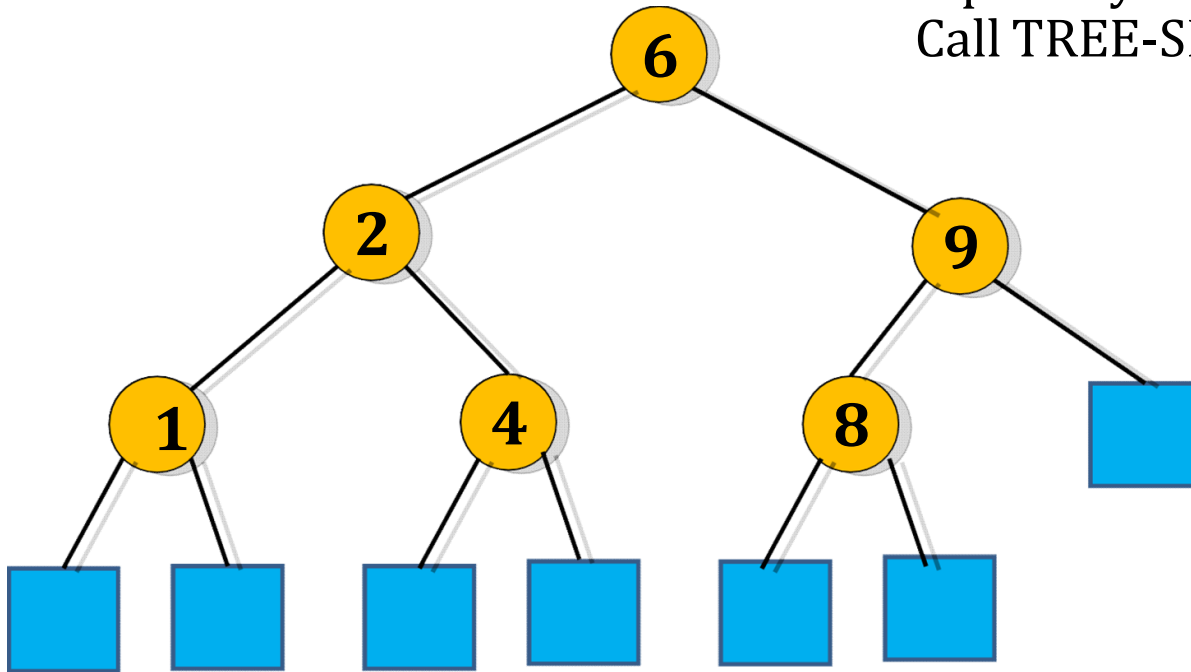
BST Search Algorithm

TREE-SEARCH(x,k)

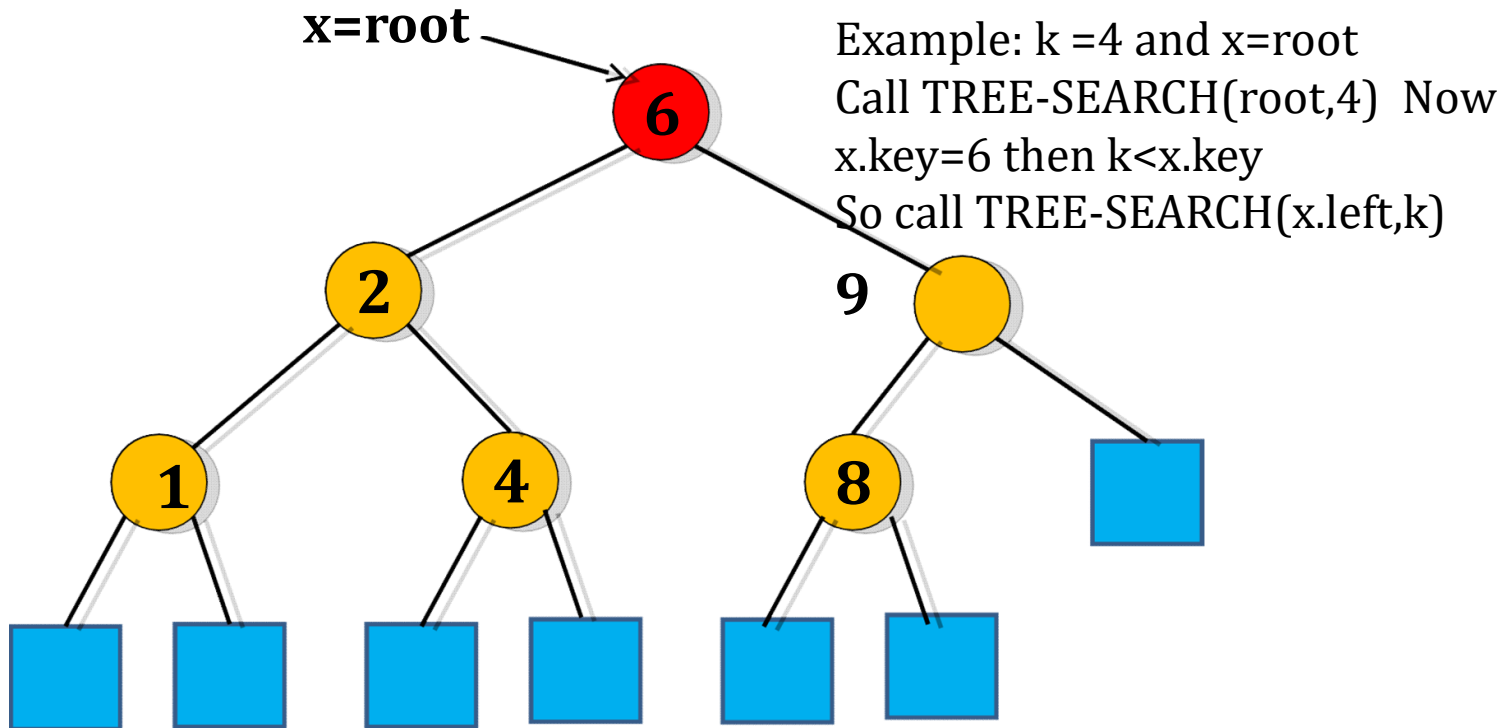
1. If $x == \text{NULL}$ or $k == x \rightarrow \text{data}$
2. return x
3. If $k < x \rightarrow \text{data}$
4. return TREE-SEARCH($x \rightarrow \text{left}$, k)
5. else
 return TREE-SEARCH($x \rightarrow \text{right}$, k)

Binary Tree Search Algorithm

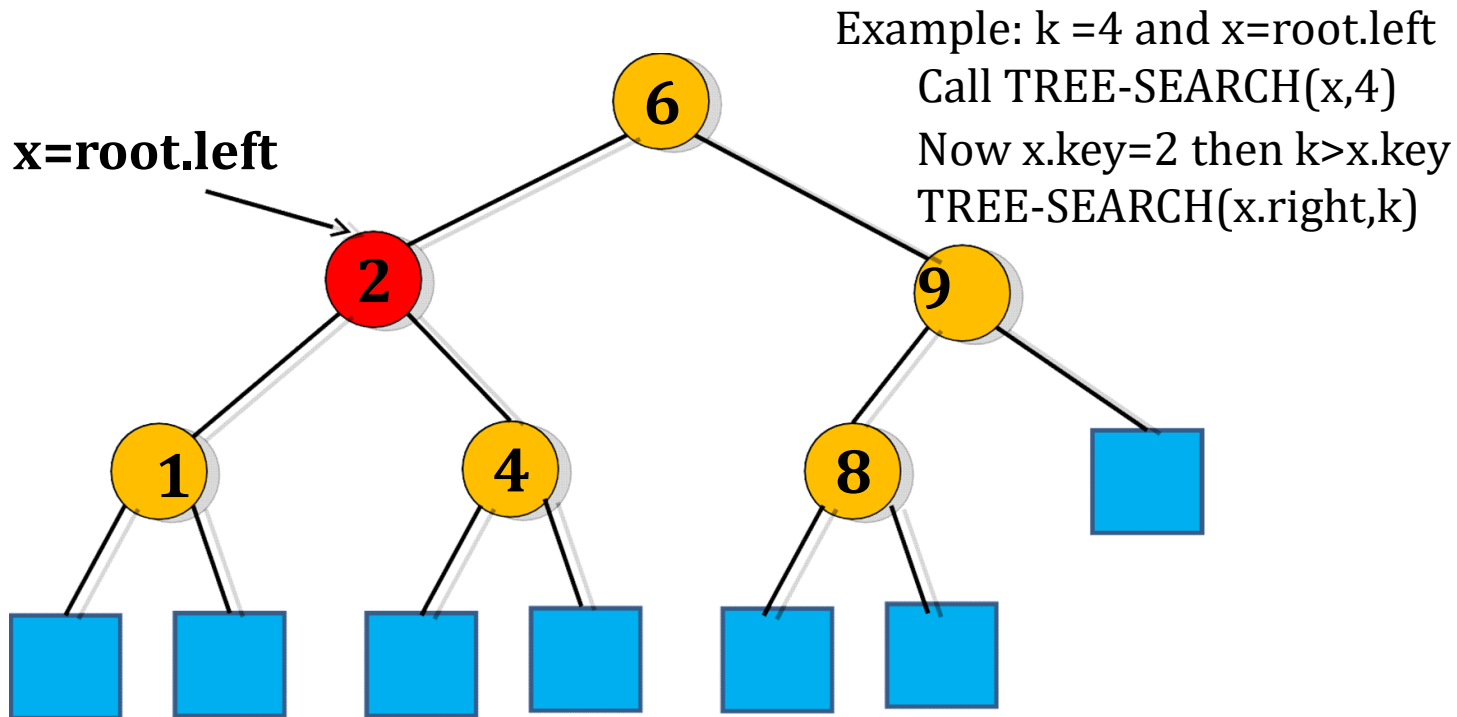
Example: key = 4 then find(4)
Call TREE-SEARCH(x,k)



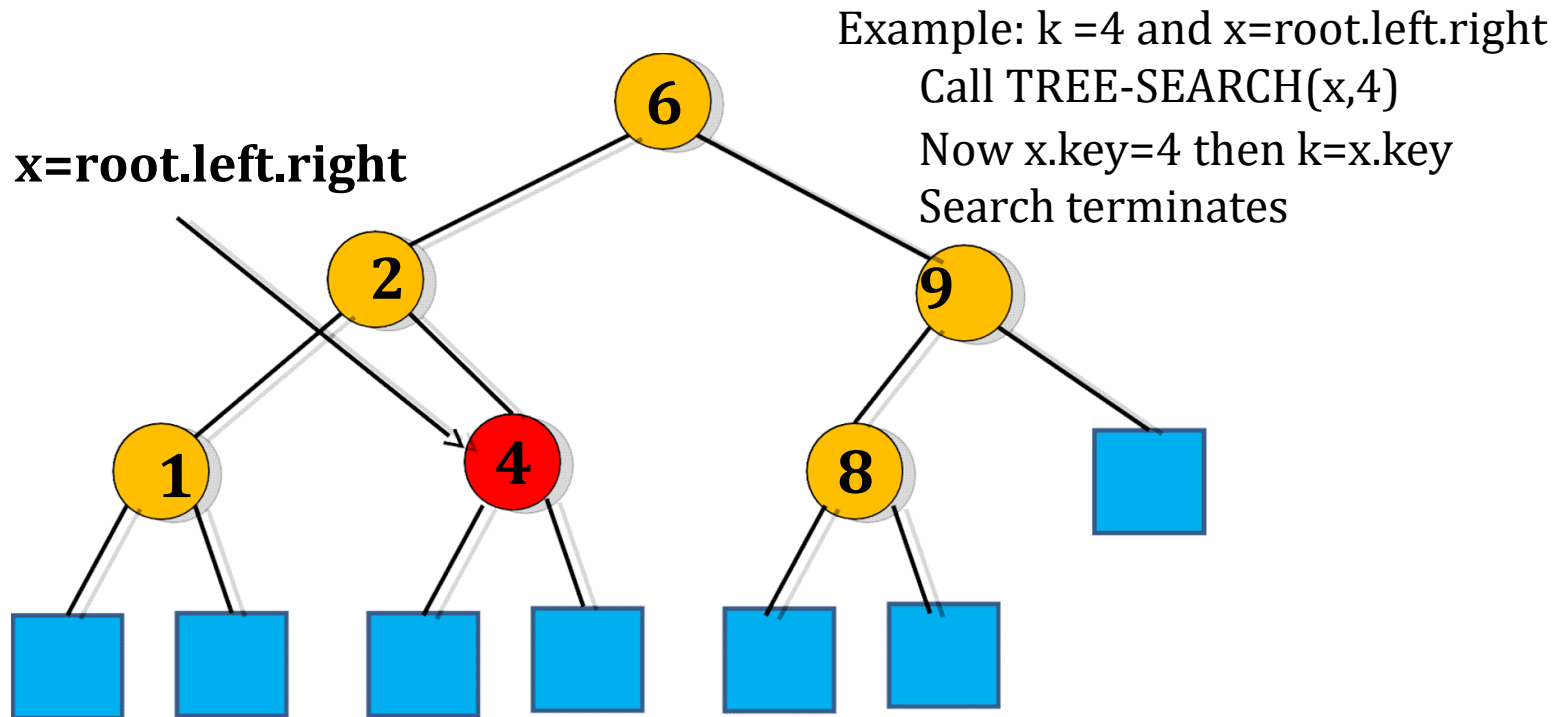
Binary Tree Search Algorithm



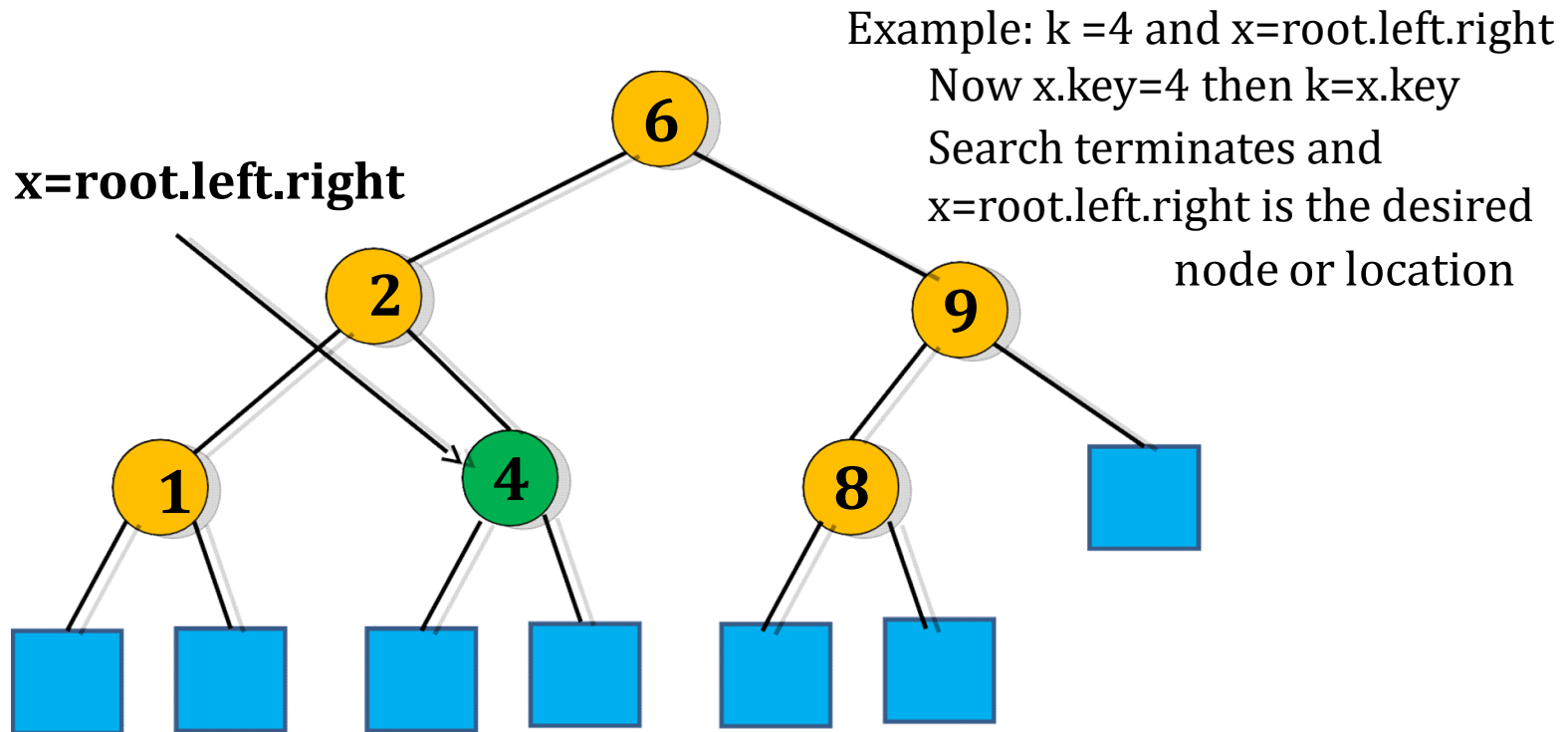
Binary Tree Search Algorithm



Binary Tree Search Algorithm



Binary Tree Search Algorithm



BST - Pseudo code

if the tree is empty
return NULL

else if the key value in the node(root) equals the target
return the node value

else if the key value in the node is greater than the target
return the result of searching the left subtree

else if the key value in the node is smaller than the target
return the result of searching the right subtree

Search in a BST: C code

```
Ptnode search(ptnode root,int key)
{
/* return a pointer to the node that
contains key. If there is no such node, return
NULL */

if (!root) return NULL;
if (key == root->key) return root;
    if (key < root->key)
return search(root->left,key) ;
    return search(root->right,key) ;
}
```

Minimum Key or Element

- We can always find an element in a binary search tree whose key is minimum by following the left children from the root until we encounter a NULL.
- Otherwise if a node x has no left subtree then the value $x.key$ contained in root x is the minimum key or element. The procedure for finding the minimum key:
 - TREE-MINIMUM(x)
 1. while $x \rightarrow \text{left} \neq \text{NULL}$
 2. $x = x \rightarrow \text{left}$
 3. return x

Maximum Key or Element

- We can always find an element in a binary search tree whose key is maximum by following the right children from the root until we encounter a NULL.
- Otherwise if a node x has no right subtree then the value $x.key$ contained in root x is the maximum key or element. The procedure for finding the maximum key:
 - TREE-MAXIMUM(x)
 1. while $x \rightarrow \text{right} \neq \text{NULL}$
 2. $x = x \rightarrow \text{right}$
 3. return x

Insert a value into the BST

- To insert a new value v into a binary search tree T , we use the procedure TREE-INSERT.
- The procedure takes a node z for which $z.key=v$, $z \rightarrow left = NULL$ and $z \rightarrow right = NULL$.
- It modifies T and some of the attributes of z in such a way that it inserts z into an appropriate position in the tree

Insert a value into the BST

- Suppose v is the value we want to insert and z is the node (New or NULL) we are supposed to find to insert the value v .
 $z.p$ denotes the parent of z .
- X is a pointer that traces a simple path downward the tree and y is the trailing pointer as the parent of x . $T.root$ denote the root of the tree.
- Now the intention is to find a new or NULL node that will satisfy the BST property after placing the value v . The procedure first consider x as the root of the tree thus the parent of the root $y=NULL$.
- In steps 3 to 7 the procedure causes the two pointer y and x to move down the tree, going left or right depending on the comparison of $z.key$ with $x.key$, until x becomes NULL.

Insert a value into the BST

- Now this NULL occupies the position z , where we wish to place the input item.
- At this time we need y the parent of the desired node. This is why at step four we always storing the parent of current node x while moving downward. At the end of step 7 (in step 8) we make this node the parent of z ($z.p$).
- From steps 9 to 11:
 - Now **if** tree is empty ($y == \text{NULL}$) then *create a root* node with the new key ($T.\text{root} = z$)
 - If the value v is less than the value of the parent ($z.\text{key} < y.\text{key}$) then make it as the left-child of the parent ($y.\text{left} = z$)
 - If the value v is greater than the value of the parent ($z.\text{key} > y.\text{key}$) then make it as the right-child of the parent ($y.\text{right} = z$)

Insert a value into the BST

▪ TREE-INSERT(T, z)

1. $y = \text{NULL}$

2. $x = T.\text{root}$

3. While $x \neq \text{NULL}$

4. $y = x$

5. if $z.\text{key} < x.\text{key}$

6. $x = x.\text{left}$

7. else $x = x.\text{right}$

8. $z.p = y$

9. if $y == \text{NULL}$

10. $T.\text{root} = z$

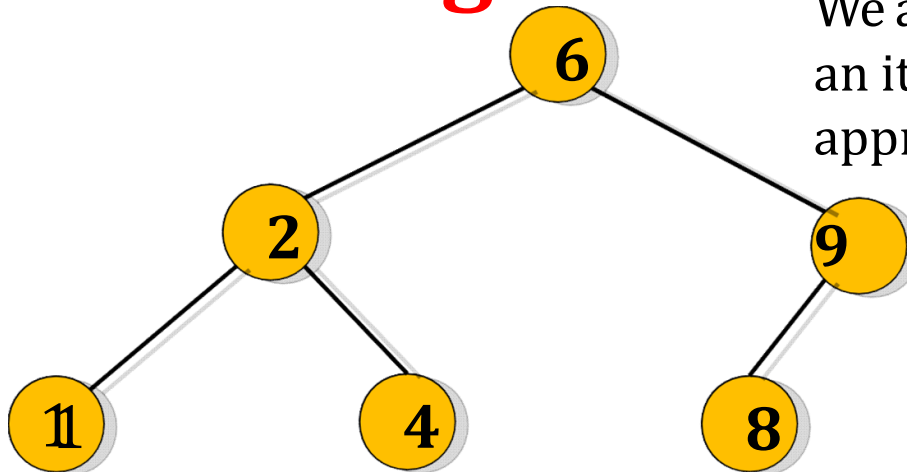
11. else if $z.\text{key} < y.\text{key}$

12. $y.\text{left} = z$

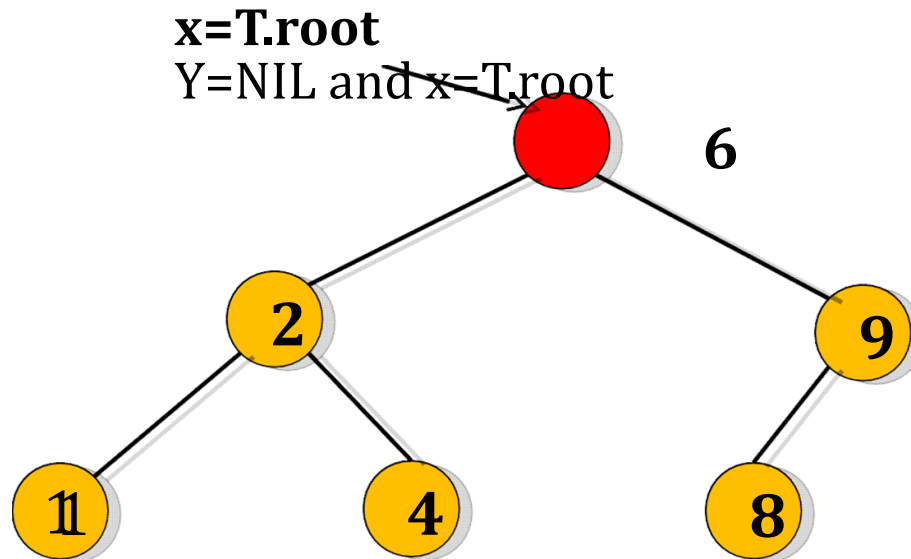
13. else $y.\text{right} = z$

BST Insertion Algorithm

We are supposed to insert
an item value 5 and find an
appropriate node z for it



BST Insertion Algorithm

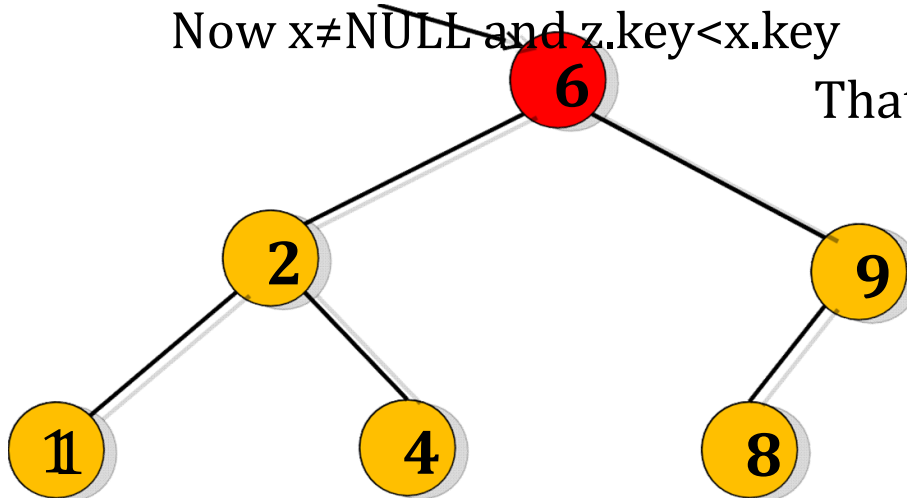


BST Insertion Algorithm

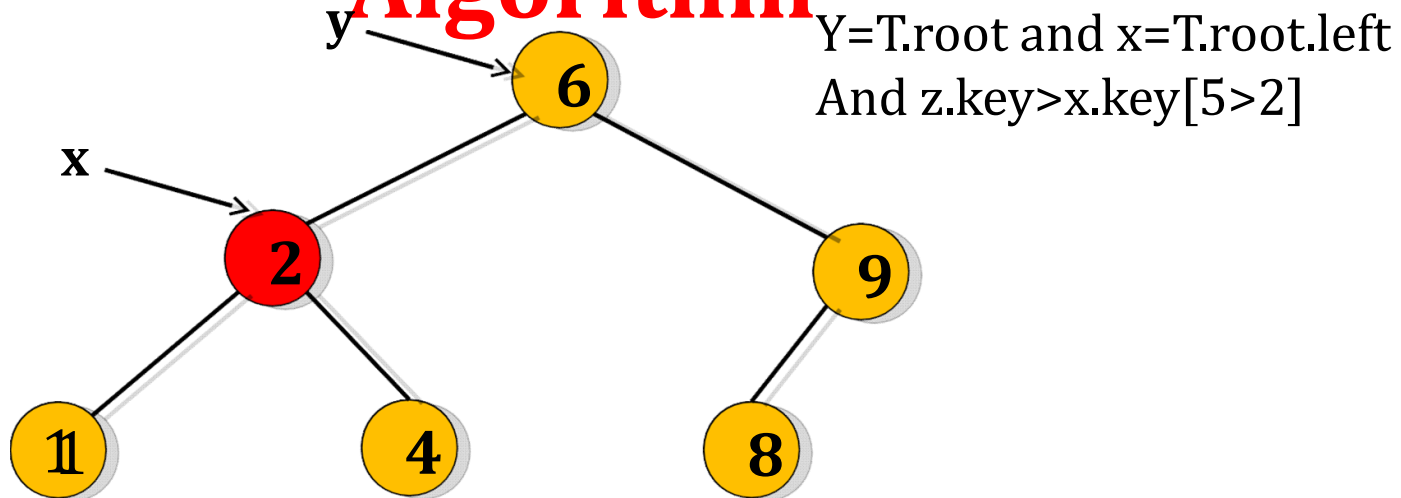
x=T.root

Now $x \neq \text{NULL}$ and $z.\text{key} < x.\text{key}$

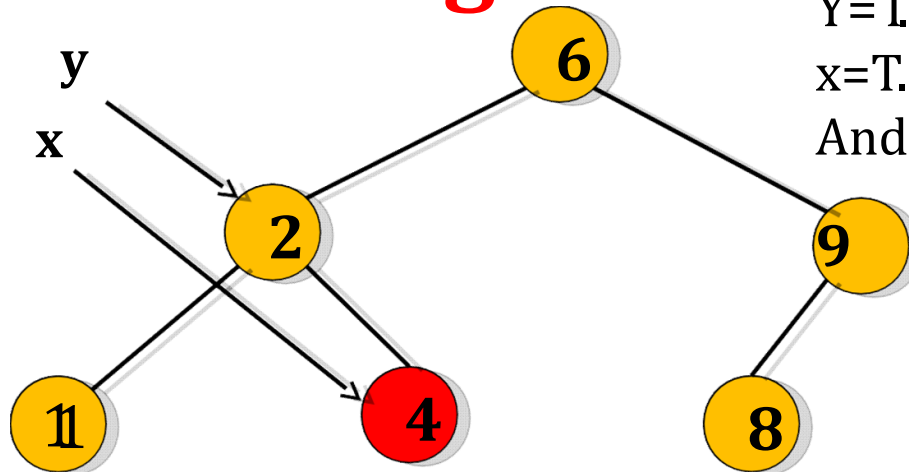
That is $[5 < 6]$



BST Insertion Algorithm



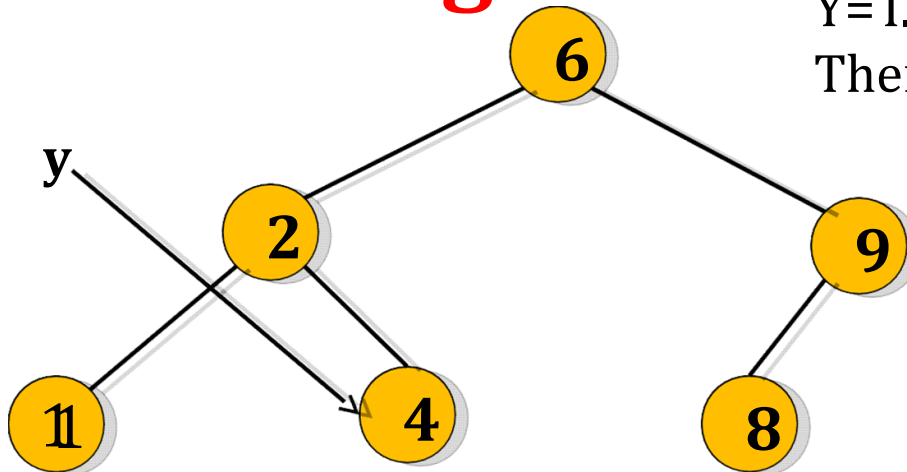
BST Insertion Algorithm



$Y = T.root.left$ and
 $x = T.root.left.right$
And $z.key > x.key [5 > 4]$

BST Insertion Algorithm

$Y = T.\text{root}.\text{left}.\text{right}$ and $x = \text{NIL}$
Then $z.p = T.\text{root}.\text{left}.\text{right}$



BST Insertion

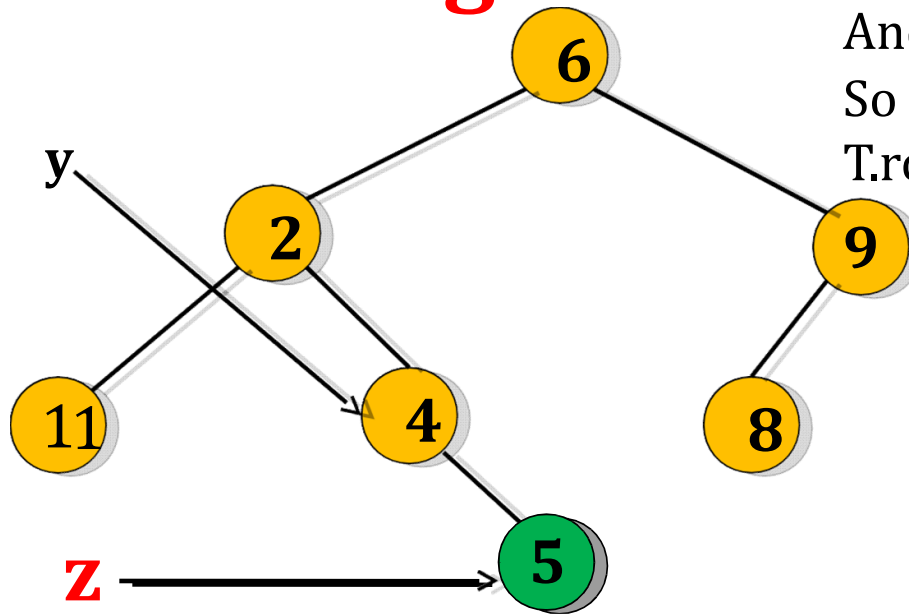
Algorithm

$Y = T.\text{root}.\text{left}.\text{right}$ [$y \neq \text{NIL}$]

And $z.\text{key} > y.\text{key}$ [$5 > 4$]

So $y.\text{right} = z$ that is

$T.\text{root}.\text{left}.\text{right}.\text{right} = z$



Insertion in BST – Pseudo code

if tree is empty

create a root node with the new key

else

compare key with the root node

if key = node key

replace the node with the new value

else if key > node key

compare key with the right subtree:

if subtree is empty *create a leaf node*

else *add key* in right subtree

else key < node key

compare key with the left subtree:

if the subtree is empty *create a leaf node*

else *add key* to the left subtree

Insertion into a BST: C code

```
void insert (ptnode *node, int key)
{
    ptnode ptr,temp;
    temp = search(*node, key);
    if (!temp) //element does not exist
    {
        ptr = (ptnode) malloc(sizeof(tnode));
        if (IS_FULL(ptr)) {
            fprintf(stderr, "The memory is full\n");
            exit(1);
        }
        ptr->key = key;
        ptr->left = ptr->right = NULL;
        if (*node)
            if (key<temp->key)
                temp->left=ptr;
            else
                temp->right = ptr;
            else *node = ptr;
    }
}
```



```
typedef struct tree_node *tree_ptr;
struct tree_node
{
int element;
tree_ptr left;
tree_ptr right;
};
typedef tree_ptr SEARCH_TREE;
```

```
tree_ptr find( element_type x, SEARCH_TREE T )
{
    if( T == NULL )
        return NULL;
    if( x < T->element )
        return( find( x, T->left ) );
    else
        if( x > T->element )
            return( find( x, T->right ) );
        else
            return T;
}
```

Figure 4.18 Find operation for binary search trees

```
tree_ptr find_min( SEARCH_TREE T )
{
    if( T == NULL )
        return NULL;
    else
        if( T->left == NULL )
            return( T );
        else
            return( find_min ( T->left ) );
}
```

```
tree_ptr find_max( SEARCH_TREE T )
{
    if( T != NULL )
        while( T->right != NULL )
            T = T->right;
    return T;
}
```

Figure 4.20 Non recursive implementation of find_max for binary search trees

```

tree_ptr insert( element_type x, SEARCH_TREE T )
{
if( T == NULL )
{ /* Create and return a one-node tree */
    T = (SEARCH_TREE) malloc ( sizeof (struct tree_node) );
    if( T == NULL )
        fatal_error("Out of space!!!");
    else
    {
        T->element = x;
        T->left = T->right = NULL;
    }
}
else if( x < T->element )
    T->left = insert( x, T->left );
else
    if( x > T->element )
        T->right = insert( x, T->right );
/* else x is in the tree already. We'll do nothing */
return T; /* Don't forget this line!! */
}

```

Figure 4.22 Insertion into a binary search tree

Delete a value from the BST

Removing a node from a BST is a bit more complex, since we do not want to create any "gap" in the tree. The intention is to **remove** the specified item from the BST and **adjusts** the tree

The binary search algorithm is used to locate the target item: **starting at the root** it probes down the tree till it finds the target or reaches a leaf node (target not in the tree)

If the node has one child then the child is spliced to the parent of the node.

If the node has two children then its successor has no left child; copy the successor into the node and delete the successor instead. $\text{TREE-DELETE}(T, z)$ removes the node pointed to by z from the tree T .

It returns a pointer to the node removed so that the node can be put on a free-node list.

The overall strategy for deleting a node or item from a binary search tree can be described through some cases.

Delete a value from the BST

Experimenting the cases:

- **if** the tree is empty return false
- **else** Attempt to locate the node containing the target using the binary search algorithm:
 - if** the target is not found return false
 - else** the target is found, then remove its node. Now while removing the node four cases may happen

Delete a value from the BST

Case 1: if the node has 2 empty subtrees

- replace the link in the parent with null

Case 2: if the node has a left and a right subtree

- replace the node's value with the max value in the left subtree
- delete the max node in the left subtree

Case 3: if the node has no left child

- link the parent of the node to the right (non-empty) subtree

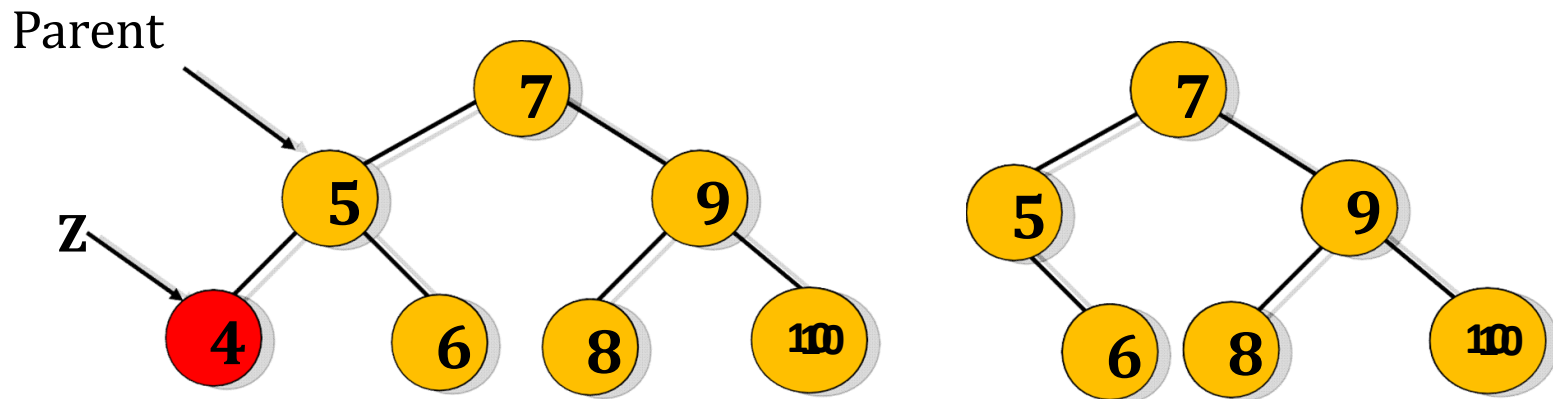
Case 4: if the node has no right child

- link the parent of the target to the left (non-empty) subtree

Delete a value from the BST

Case 1: removing a node with 2 EMPTY SUBTREES
-replace the link in the parent with null

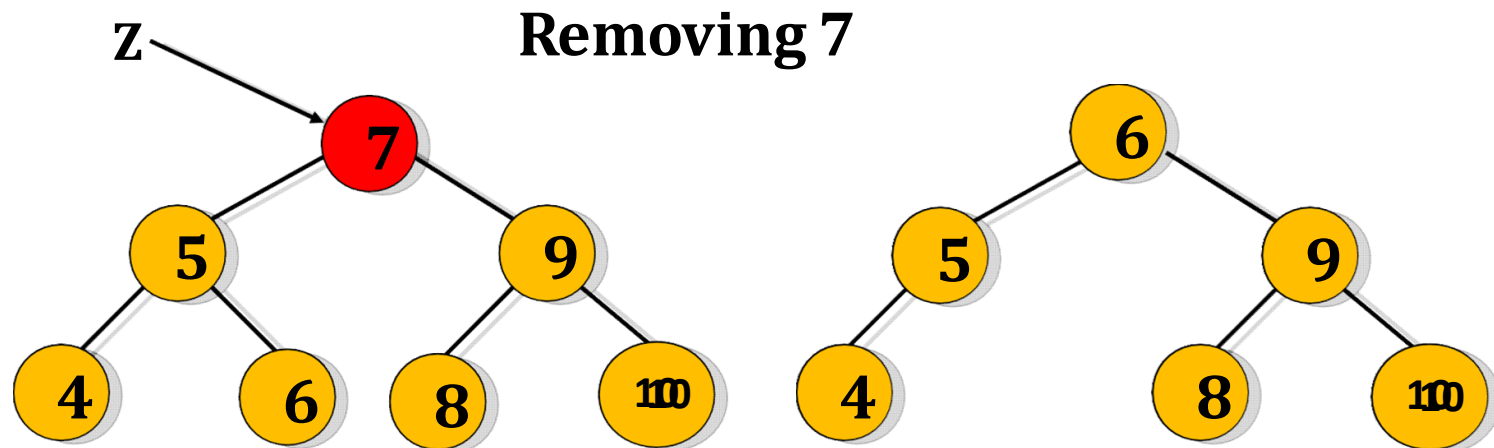
Removing 4



Delete a value from the BST

Case 2: removing a node with 2 SUBTREES

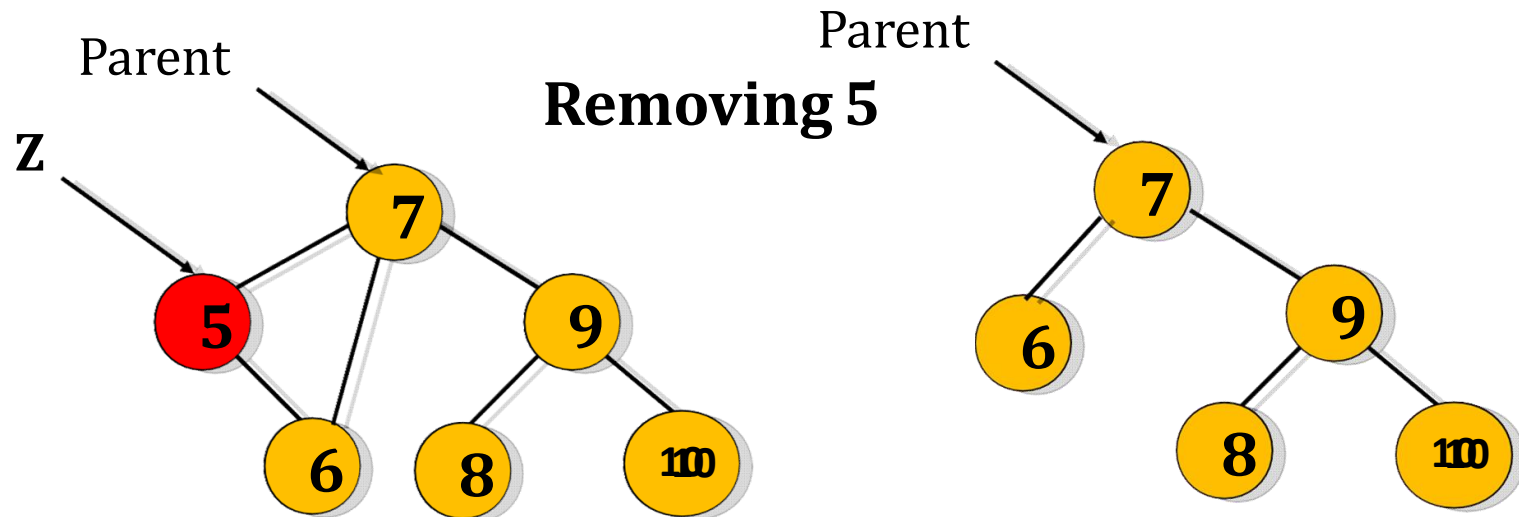
- replace the node's value with the max value in the left subtree
- delete the max node in the left subtree



Delete a value from the BST

Case 3: if the node has no left child

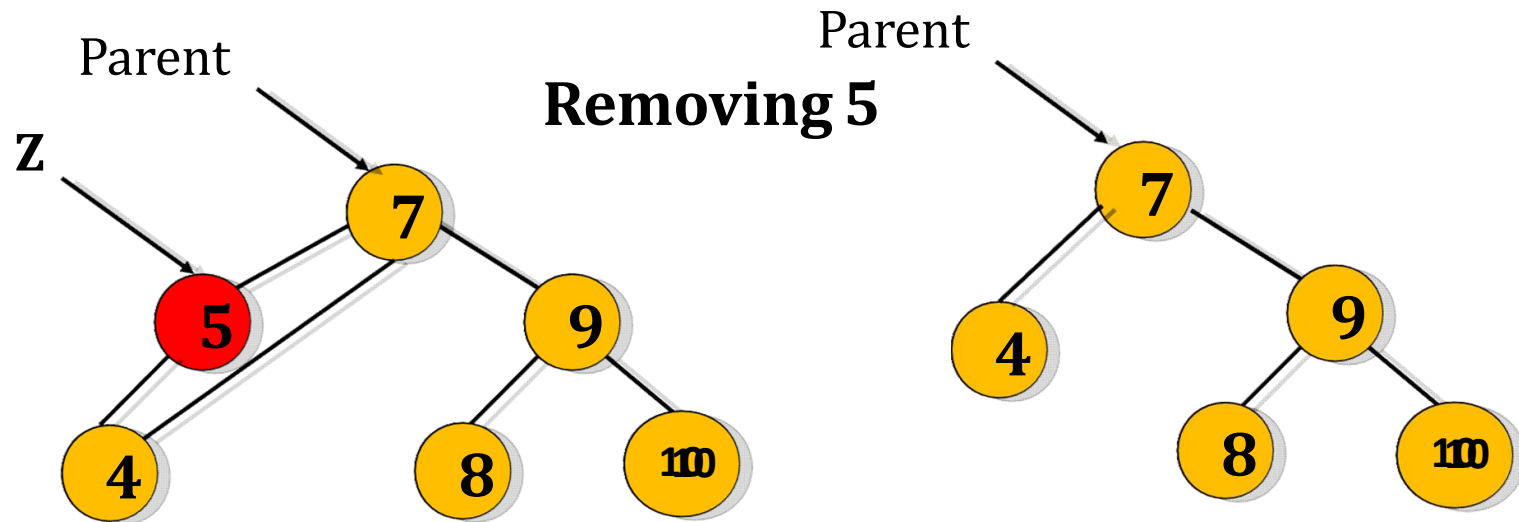
- link the parent of the node to the right (non-empty) subtree



Delete a value from the BST

Case 4: if the node has no right child

- link the parent of the node to the left (non-empty) subtree



Deletion algorithm

```
tree_ptr delete( element_type x, SEARCH_TREE T )
{
    tree_ptr tmp_cell, child;
    if( T == NULL )
        error("Element not found");
    else
        if( x < T->element ) /* Go left */
            T->left = delete( x, T->left );
        else
            if( x > T->element ) /* Go right */
                T->right = delete( x, T->right );
            else /* Found element to be deleted */
                if( T->left && T->right ) /* Two children */
                { /* Replace with smallest in right subtree */
                    tmp_cell = find_min( T->right );
                    T->element = tmp_cell->element;
                    T->right = delete( T->element, T->right );
                }
            else /* One child */
                }
```

```
tmp_cell = T;
if( T->left == NULL ) /* Only a right child */
child = T->right;
if( T->right == NULL ) /* Only a left child */
child = T->left;
free( tmp_cell );
return child;
}
return T;
}
```

Figure 4.25 Deletion routine for binary search trees

Analysis of BST Operations

- The complexity of operations **search**, **insert** and **remove** in BST is $O(h)$, where h is the height.
- When the tree is balanced then it is $O(\log n)$.
The updating operations cause the tree to become unbalanced.
- The tree can degenerate to a linear shape and the operations will become $O(n)$

Declaration of a node

```
struct node
{
    int data;
    struct node *right_child;
    struct node *left_child;
};
```

Creation of a node

```
struct node* new_node(int x)
{
    struct node *p;
    p = malloc(sizeof(struct node));
    p->data = x;
    p->left_child = NULL;
    p->right_child = NULL;

    return p;
}
```

Insertion of a node

```
struct node* insert(struct node *root, int x)
{
    if(root==NULL)
        return new_node(x);
    else if(x>root->data)
        root->right_child = insert(root->right_child, x);
    else
        root->left_child = insert(root->left_child,x);
    return root;
}
```

Deletion of a node

```
struct node* delete(struct node *root, int x)
{
    if(root==NULL)
        return NULL;
    if (x>root->data)
        root->right_child = delete(root->right_child, x);
    else if(x<root->data)
        root->left_child = delete(root->left_child, x);
}
```

Deletion of Leaf Node

```
else
{
    //No Children
    if(root->left_child==NULL &&
        root->right_child==NULL)
    {
        free(root);
        return NULL;
    }
}
```

Node with One Child

```
else if(root->left_child==NULL ||
        root->right_child==NULL)
{
    struct node *temp;
    if(root->left_child==NULL)
        temp = root->right_child;
    else
        temp = root->left_child;
    free(root);
    return temp;
}
```

Node with two Children

```
else
{
    struct node *temp = find_minimum(root->right_child);
    root->data = temp->data;
    root->right_child = delete(root->right_child, temp->data);
}
}
return root;
}
```

Minimum Element in BST

```
struct node* find_minimum(struct node *root)
{
    if(root == NULL)
        return NULL;
    else if(root->left_child != NULL)
        return find_minimum(root->left_child);
    return root;
}
```


Kth Minimum Element in BST

```
int count = 0;
Struct node* kthSmallest(struct node*
root, int k)
{
    if (root == NULL)
        return NULL;

    struct node* left = kthSmallest(root->left, k);
    if (left != NULL)
        return left;
```

```
        count++;
        if (count == k)
            return root;

        return kthSmallest(root->right, k);
}
```

Kth Maximum Element in BST

```
int count = 0;
Struct node* kthlargest(struct node*
root, int k)
{
    if (root == NULL)
        return NULL;

    struct node* right = kthSmallest(root->right,
k);
    if (right != NULL)
        return right;

    count++;
    if (count == k)
        return root;

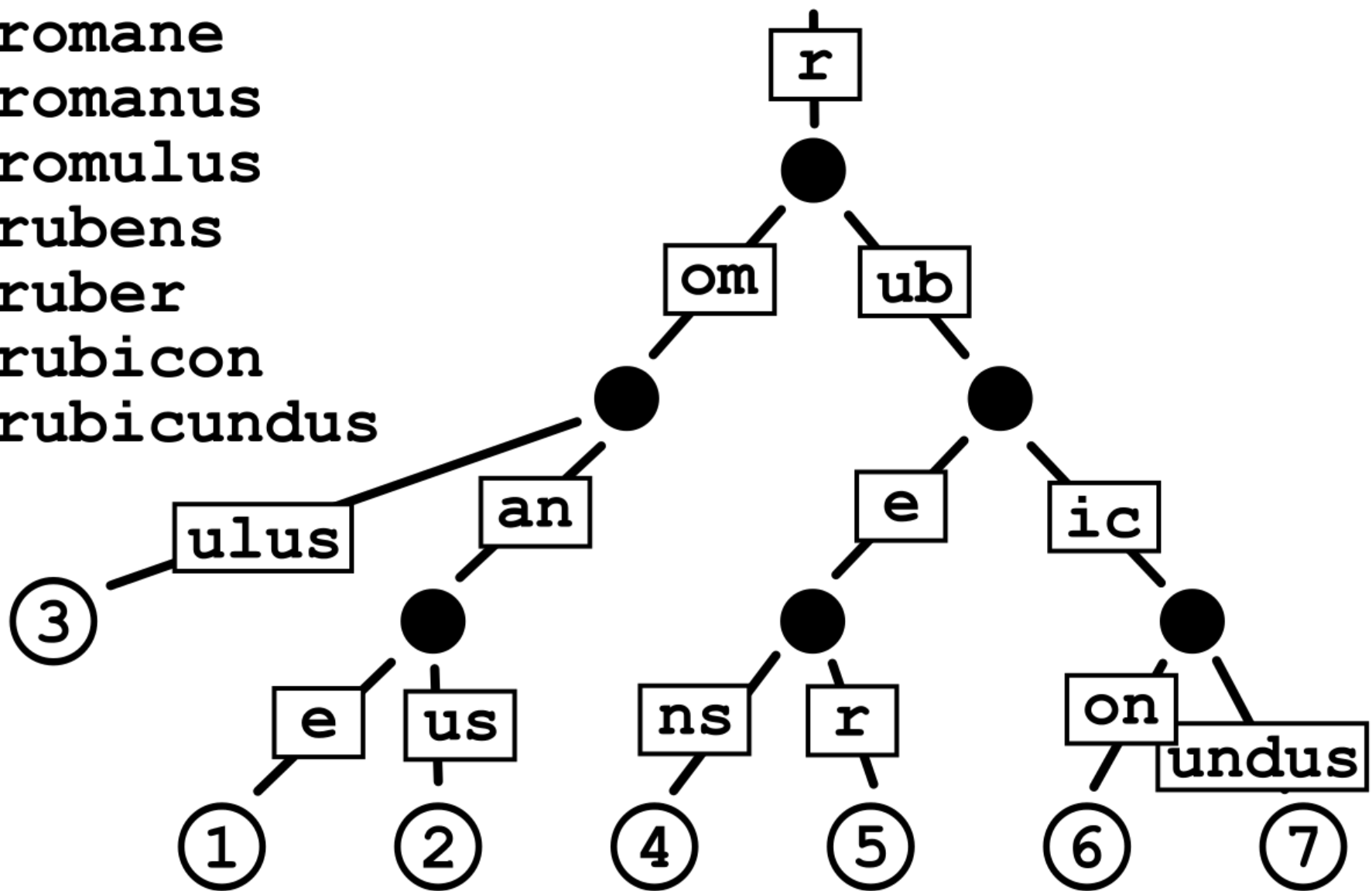
    return kthlargest(root->left, k);
}
```

Applications of binary trees

- **Binary Search Tree** - Used in many search applications where data is constantly entering/leaving, such as the map and set objects in many languages' libraries.
- **Binary Space Partition** - Used in almost every 3D video game to determine what objects need to be rendered.
- **Binary Tries** - Used in almost every high-bandwidth router for storing router-tables.
- **Hash Trees** - Used in torrents and specialized image-signatures in which a hash needs to be verified, but the whole file is not available. Also used in blockchains for eg. Bitcoin.
- **Heaps** - Used in implementing efficient priority-queues, which in turn are used for scheduling processes in many operating systems, Quality-of-Service in routers, and A* (path-finding algorithm used in AI applications, including robotics and video games). Also used in heap-sort.

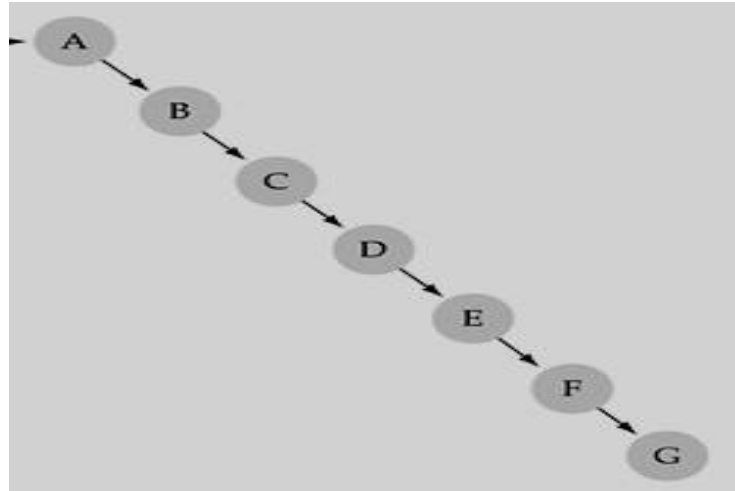
- **Huffman Coding Tree** - Used in compression algorithms, such as those used by the .jpeg and .mp3 file-formats.
- **GGM Trees** - Used in cryptographic applications to generate a tree of pseudo-random numbers.
- **Syntax Tree** - Constructed by compilers and (implicitly) calculators to parse expressions.
- **Treap** - Randomized data structure used in wireless networking and memory allocation.

- 1 romane
- 2 romanus
- 3 romulus
- 4 rubens
- 5 ruber
- 6 rubicon
- 7 rubicundus



What is a Degenerate BST?

- A degenerate binary search tree is one where most or all of the nodes contain only one sub node.
- It is unbalanced and, in the worst case, performance degrades to that of a linked list.
- If your add node function does not handle rebalancing, then you can easily construct a degenerate tree by feeding it data that is already sorted.

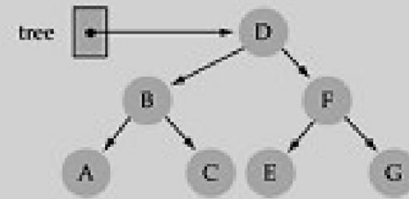


Does the order of inserting elements into a tree matter?

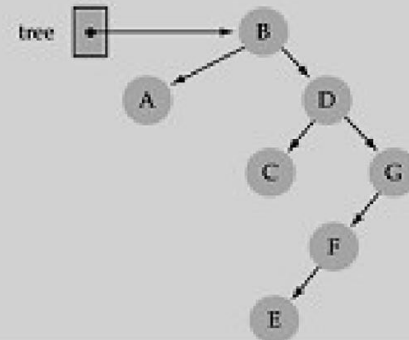
- Yes, certain orders might produce very unbalanced trees or degenerated trees!
- Unbalanced trees are not desirable because search time increases!
- Advanced tree structures, such as **red-black trees**, **AVL trees**, guarantee balanced trees.

**Does the
order of
inserting
elements
into a tree
matter?**

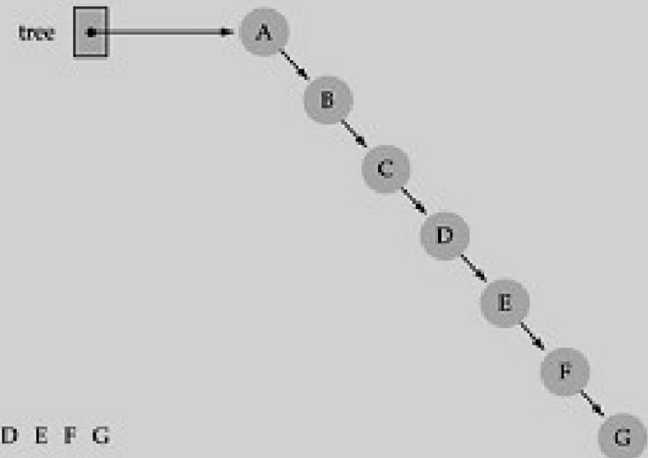
(a) Input: D B F A C E G



(b) Input: B A D C G F E



(c) Input: A B C D E F G



Better Search Trees

Prevent the degeneration of the BST :

- A BST can be set up to maintain balance during updating operations (insertions and removals)
- Types of ST which maintain the optimal performance in other words balanced trees:
 - splay trees
 - AVL trees
 - 2-4 Trees
 - Red-Black trees
 - B-trees

Operations on binary tree

Traversing : - It means processing the tree in such way that each node is visited only once

Types of traversing

- In-order traversal :- Gives us the expression in form of infix

- Pre-order traversal :- Gives us the expression in form of Pre-fix

- Post-order traversal :- Gives us the expression in postfix form

- Converse in-order traversal

- Converse Pre-order traversal

- Converse Post-order traversal

- Level order traversal

Inorder $\rightarrow$ left-root-right

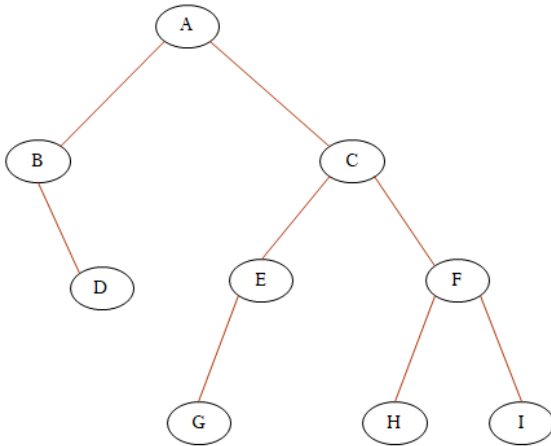
Preorder $\rightarrow$ root-left-right

Postorder $\rightarrow$ left-right-root

Converse Inorder $\rightarrow$ right-root-left

Converse Preorder $\rightarrow$ root-right-left

Converse Postorder $\rightarrow$ right-left-root



Preorder(root)

```
void preorder (struct node *root)
{
    if(root == 0)
        return;
    printf("%d",root→data);
    preorder (root→left);
    preorder(root→right);
}
```

Postorder(root)

```
void postorder (struct node *root)
{
    if(root == 0)
        return;
    postorder (root→left);
    postorder(root→right);
    printf("%d",root→data);
}
```

Inorder(root)

```
void inorder (struct node *root)
{
    if(root == 0)
        return;
    inorder (root→left);
    printf("%d",root→data);
    inorder(root→right);
}
```

Tree Traversals with an Application

- One of the application is directory structure in many common operating systems

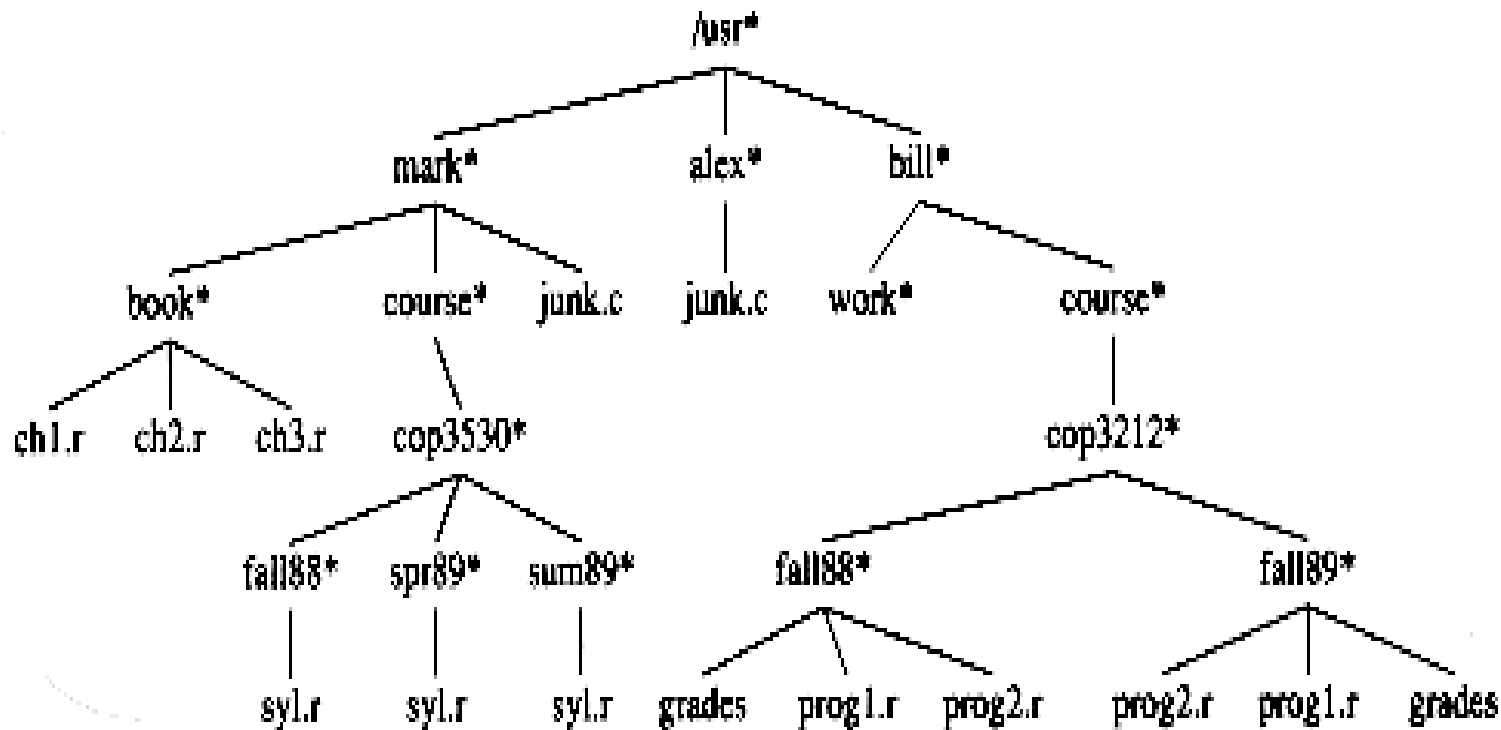


Figure 4.5 Unix directory

```

Void list_directory (
Directory_or_file D )
{
list_dir ( D, 0 );
}

void list_dir (
Directory_or_file D, unsigned int
depth )
{
/*1*/ if ( D is a legitimate entry)
{
/*2*/ print_name ( depth, D );
/*3*/ if( D is a directory )
/*4*/ for each child, c, of D
/*5*/ list_dir( c, depth+1 );
}
}

```

Figure 4.6

- we would like to list the names of all of the files in the directory.
- Our output format will be that files that are depth d will have their names indented by d tabs.
- Our algorithm is given in Figure 4.6.
- This traversal strategy is known as a ***preorder traversal***.
- if there are n file names to be output, then the running time is $O(n)$.


```
/usr
mark
book
chr1.c
chr2.c
chr3.c
course
cop3530
fall88
syl.r
spr89
syl.r
sum89
syl.r
junk.c
alex
junk.c
bill
work
course
cop3212
fall88
grades
prog1.r
prog2.r
fall89
prog1.r
prog2.r
grades
```

Figure 4.7 The (preorder) directory listing

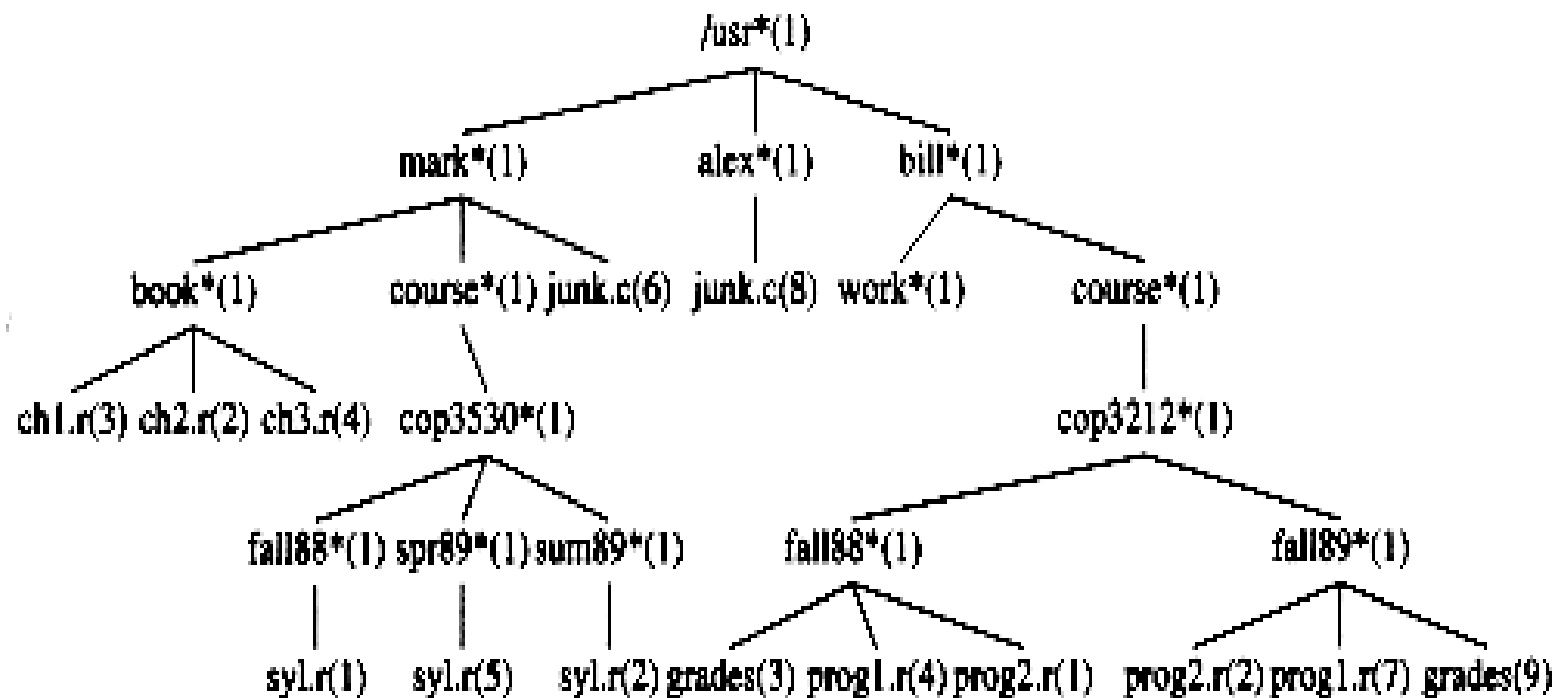


Figure 4.8 Unix directory with file sizes obtained via postorder traversal

- Since the directories are themselves files, they have sizes too.
- we would like to calculate the total number of blocks used by all the files in the tree.
- The most natural way to do this would be to find the number of blocks contained in the subdirectories */usr/mark* (30), */usr/alex* (9), and */usr/bill* (32).
- The total number of blocks is then the total in the subdirectories (71) plus the one block used by */usr*, for a total of 72.
- The function *size_directory* in Figure 4.9 implements this strategy.

```
unsigned int size_directory( Directory_or_file D )
{
    unsigned int total_size;
    /*1*/ total_size = 0;
    /*2*/ if( D is a legitimate entry)
    {
        /*3*/ total_size = file_size( D );
        /*4*/ if( D is a directory )
        /*5*/ for each child, c, of D
        /*6*/ total_size += size_directory( c );
    }
    /*7*/ return( total_size );
}
```

Figure 4.9 Routine to calculate the size of a directory (Postorder Traversal)

Postorder traversal output

ch1.r	3		
ch2.r	2		syl.r 2
ch3.r	4		sum89 3
book	10		cop3530 12
syl.r	1		course 13
fall88	2		junk.c 6
syl.r	5	mark	30
spr89	6		junk.c 8
		alex	9
		work	1
			grades 3
			prog1.r 4
			prog2.r 1
			fall88 9
			prog2.r 2
			prog1.r 7
			grades 9
			fall89 19
			cop3212 29
		course	30
		bill	32
		/usr	72

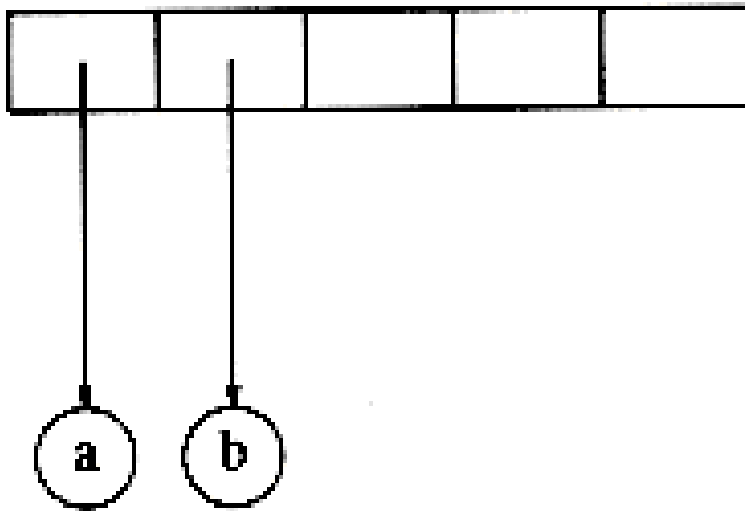
Constructing an Expression Tree

- algorithm to convert a postfix expression into an expression tree
 1. read given **postfix expression** one symbol at a time.
 2. If the symbol is an operand, we create a one-node tree and push a pointer to it onto a stack.
 3. If the symbol is an operator, we pop pointers to two trees T1 and T2 from the stack (T1 is popped first) and form a new tree whose root is the operator and whose left and right children point to T2 and T1 respectively.
 4. A pointer to this new tree is then pushed onto the stack.

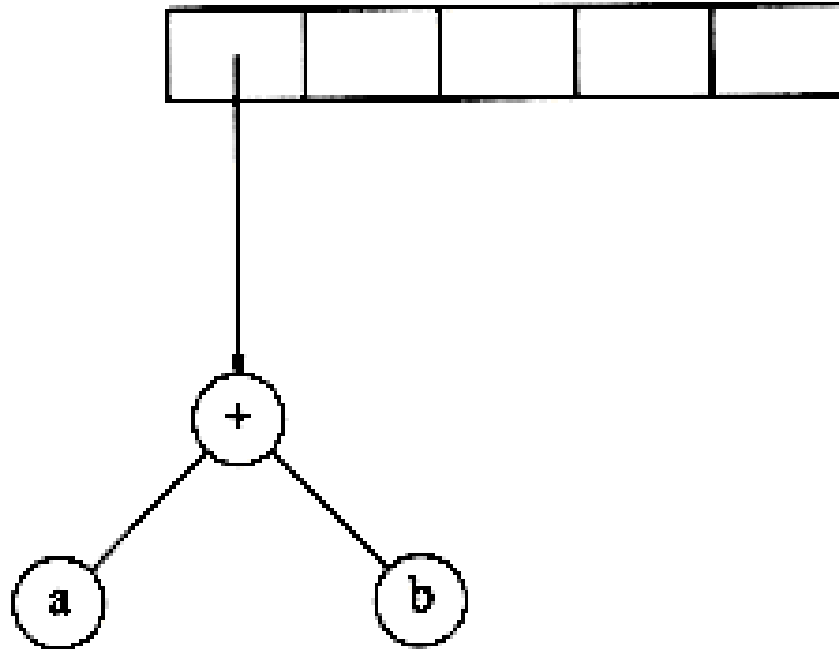
As an example, suppose the input is

$a\ b\ +\ c\ d\ e\ +\ *\ *$

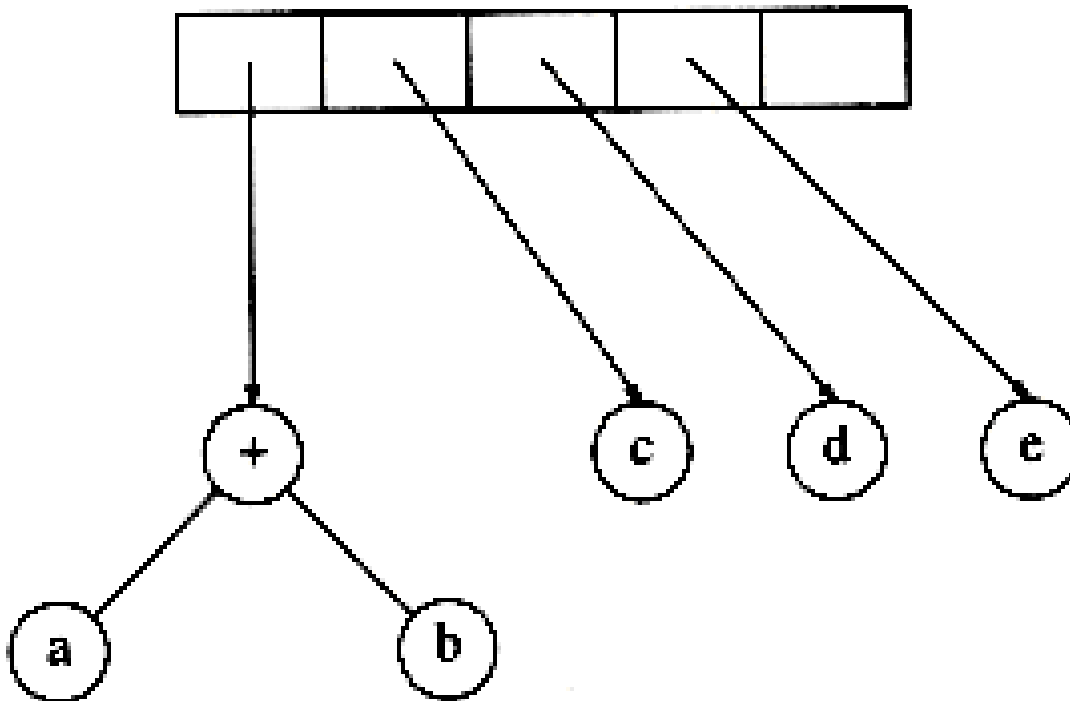
- The first two symbols are operands, so we create one-node trees and push pointers to them onto a stack



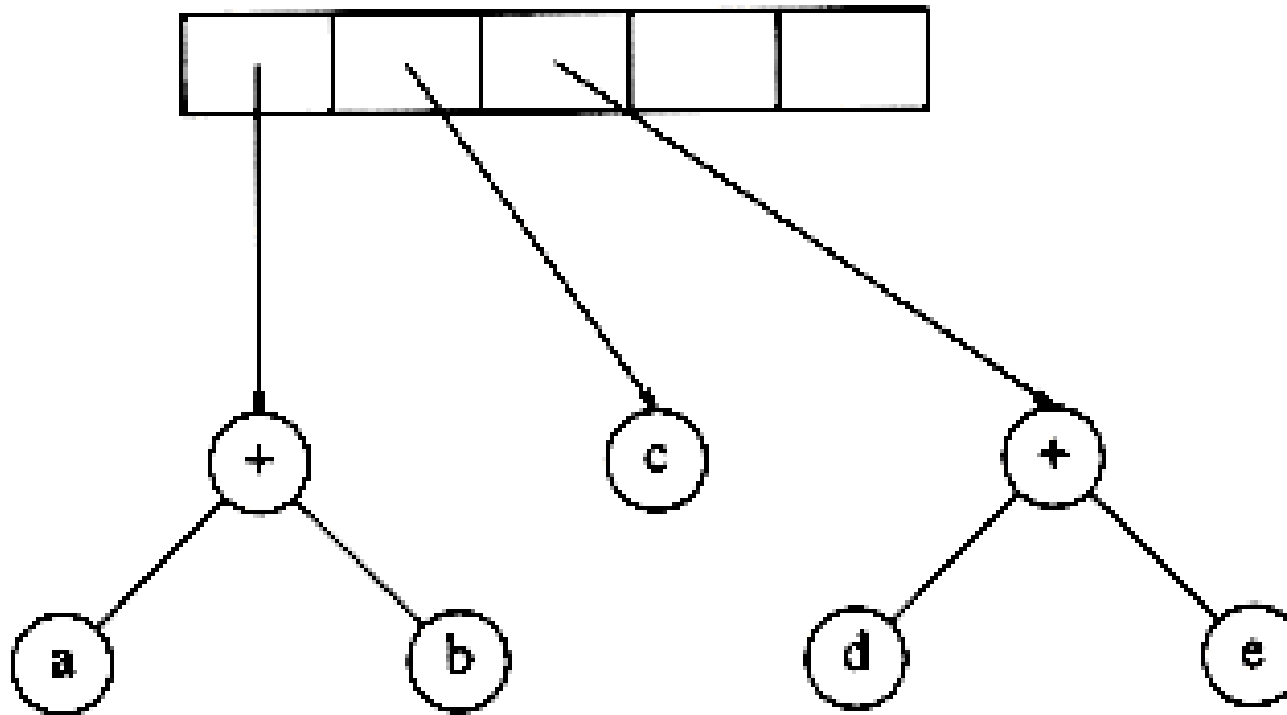
- Next, a '+' is read, so two pointers to trees are popped, a new tree is formed, and a pointer



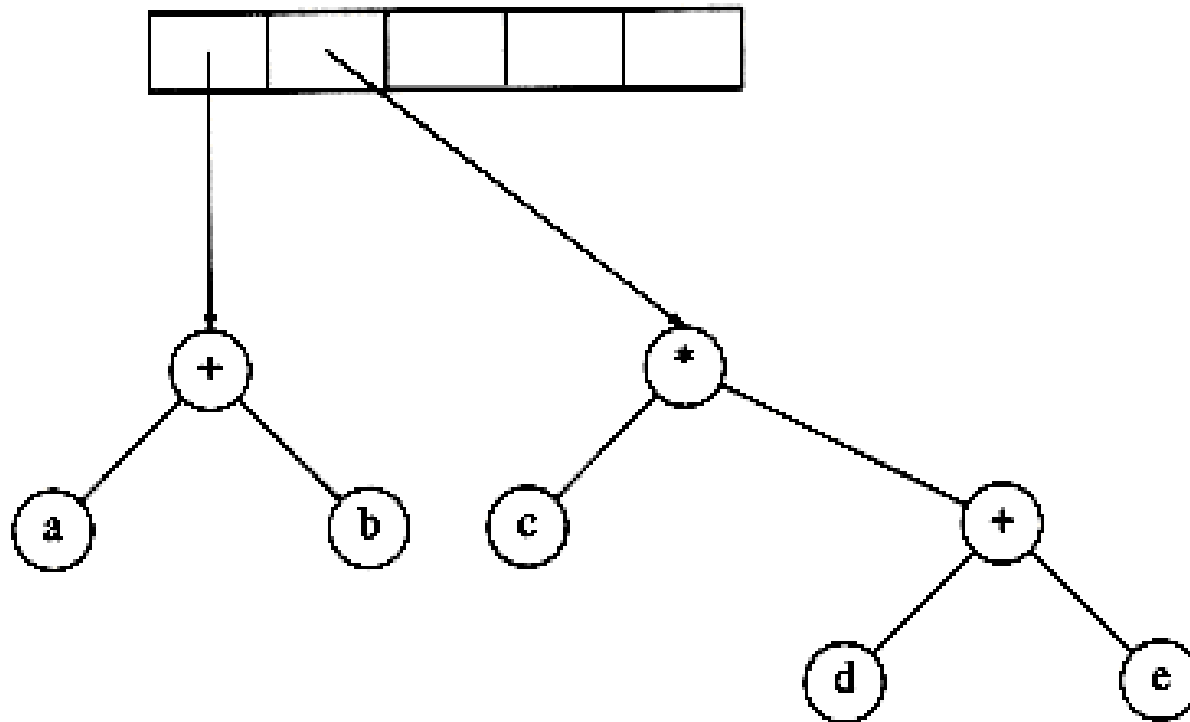
Next, c , d , and e are read, and for each a one-node tree is created and a pointer to the corresponding tree is pushed onto the stack.



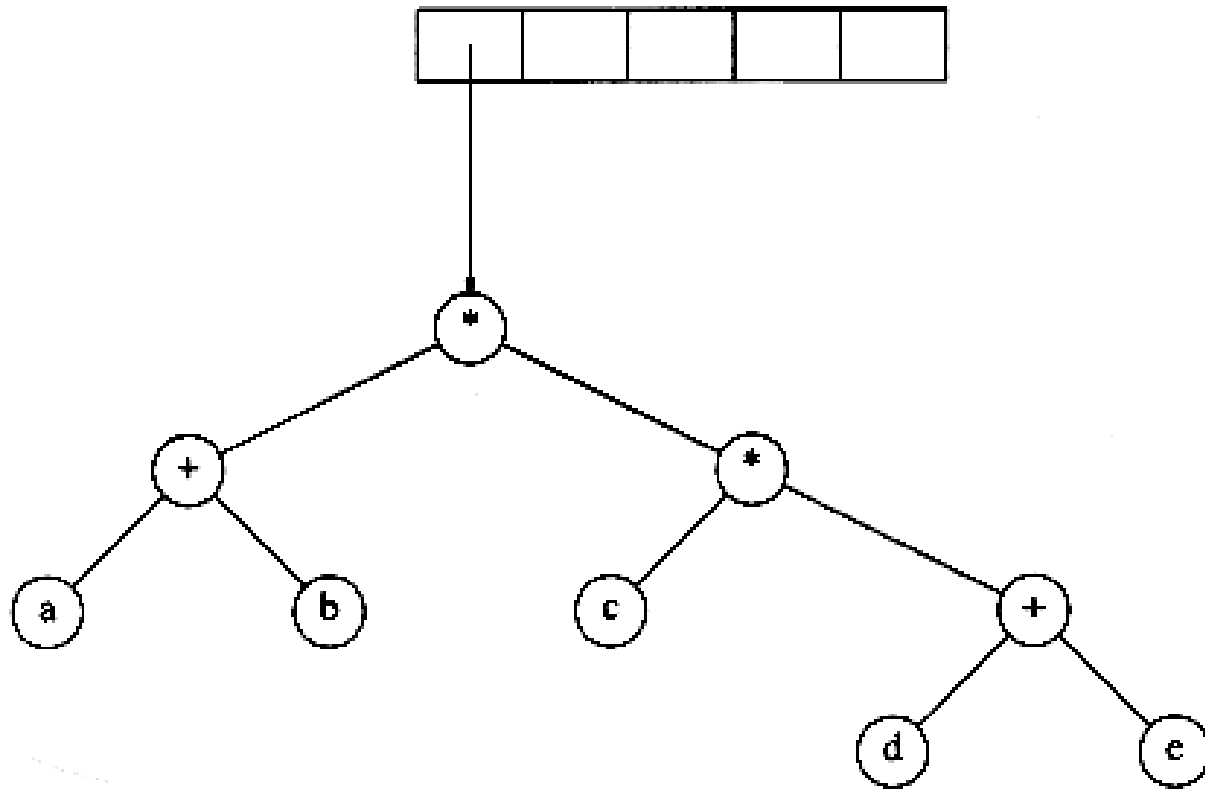
Now a '+' is read, so two trees are merged.

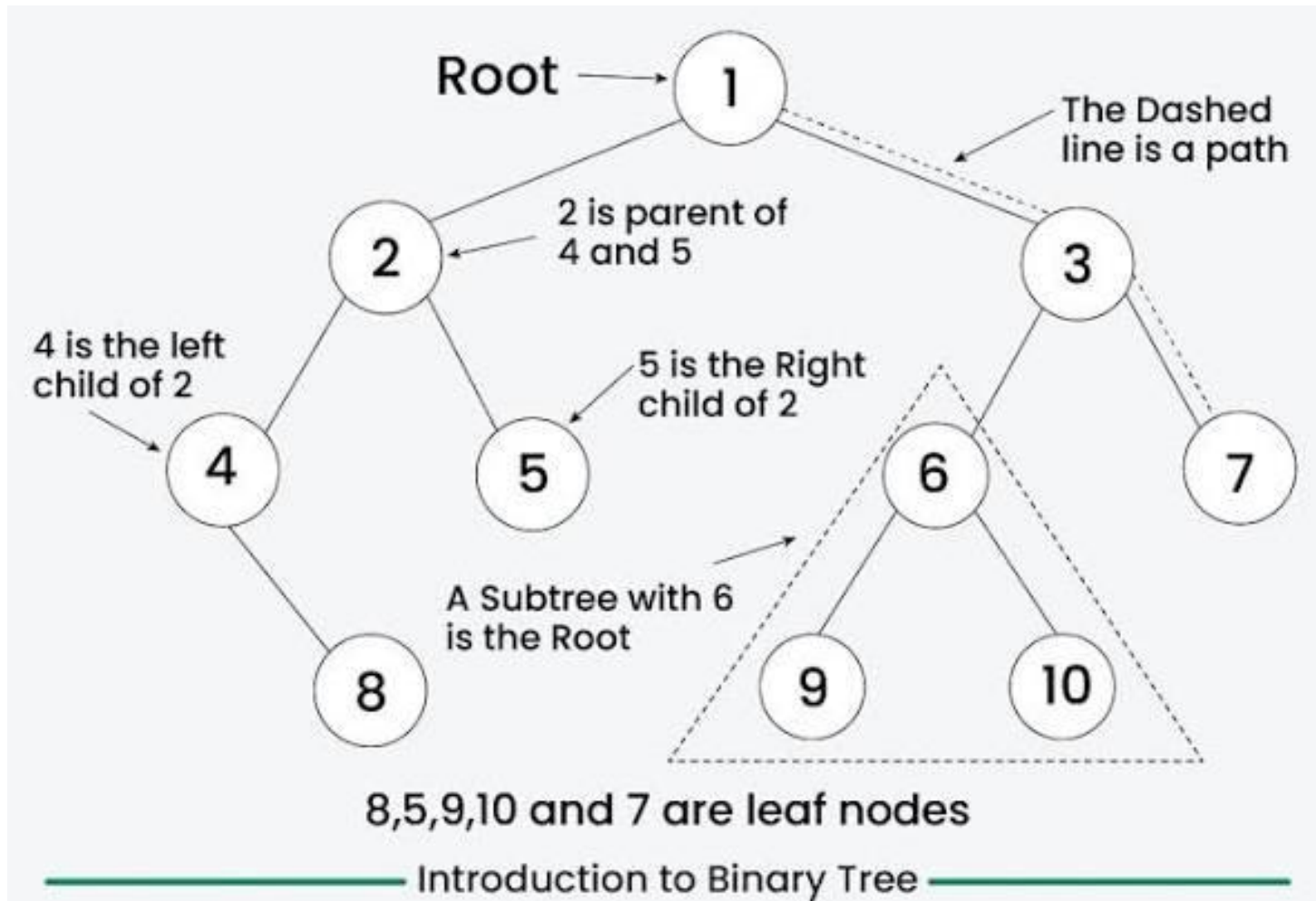


Continuing, a '*' is read, so we pop two tree pointers and form a new tree with a '*' as root.

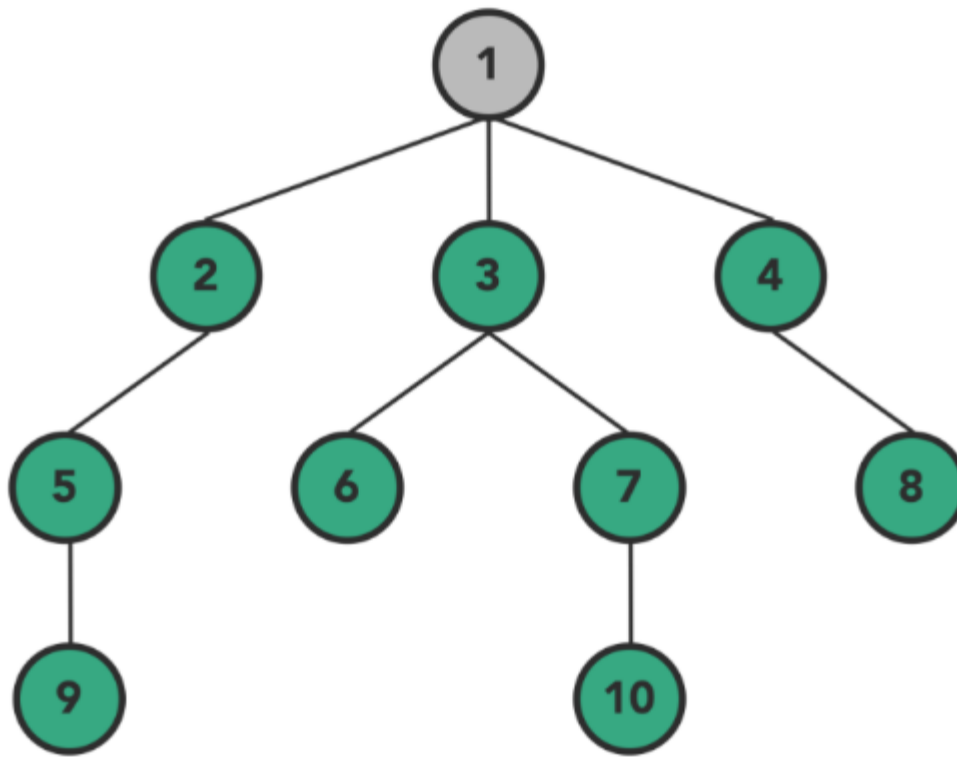


Finally, the last symbol is read, two trees are merged, and a pointer to the final tree is left on the stack.





Apply all traversals for the above binary tree



Apply all traversals for the above tree

Create Expression Tree

1) $A B + C D - * E F G * + /$

2) $A B + C D + * E F G + ^ -$

3) $A * (B + (C * (D - E)))^F$